

GrainLayer Programmer's Manual

Building a Paper Layer, One Win32 Version at a Time

A deliberately old-fashioned programming manual for a deliberately small Windows program.

NOTE: READ THIS FIRST

This edition documents the complete evolution of GrainLayer from version 0.1, when the project was still called Deckle Windows, through version 1.4.1. It explains the architecture, the important Win32 mechanisms, the bugs encountered during development, and the reasoning behind the fixes. Complete historical source listings are included in the appendices.

Page numbers are intentionally omitted. The manual uses numbered sections, monospaced source listings, ASCII diagrams, boxed notes, and current application screenshots in the spirit of 1990s programming books.

Contents

- 0.1. Building a Paper Layer, One Win32 Version at a Time
- 0.2. Preface
- 0.3. Current Build: Visual Reference
 - 0.3.1. The same desktop, without and with the overlay
 - 0.3.2. The tray interface
- 0.4. How to Read This Manual
- 1. Part I --- The First Window
 - 1.1. Chapter 0.1 --- Deckle Windows v0.1
 - 1.2. Chapter 1.2.6 --- GrainLayer v1.2.6
 - 1.3. 1. The Invisible Owner Window
 - 1.4. 2. The Layered Window
 - 1.5. 3. Premultiplied Alpha
 - 1.6. 4. Click-Through Behaviour
- 2. Part III --- The Tray Application
 - 2.1. 1. Why a Tray Icon?
 - 2.2. 2. Command IDs
 - 2.3. 3. Hotkeys
- 3. Part IV --- Building a Slider Inside a Win32 Menu
 - 3.1. 1. Owner-Drawn Menu Items
 - 3.2. 2. Finding the Item Rectangle
 - 3.3. 3. Click-to-Position
 - 3.4. 4. Dragging
 - 3.5. 5. Why Hover Does Nothing
- 4. Part V --- Persistent State
 - 4.1. 1. Where the Settings Live
 - 4.2. 2. What Is Stored
 - 4.3. 3. INI Storage
 - 4.4. 4. Windows Startup
- 5. Part VI --- Resources and the Executable
- 6. Part VII --- The Visual Studio Project
- 7. Part VIII --- What the Bugs Taught
 - 7.1. The Black Overlay
 - 7.2. Missing Shell Declarations
 - 7.3. Menu Position vs. Command ID
 - 7.4. Cascading C++ Errors
 - 7.5. Mouse Dragging Inside a Popup Menu
 - 7.6. Slider Synchronization
- 8. Part IX --- Final Behaviour
- 9. Part X --- A Short Tour of the Final Source
- 10. Appendix A --- Complete Historical Source Listings
 - 10.1. Deckle Windows v0.1

- 10.2. Deckle Windows v0.2
- 10.3. Deckle Windows v0.3
- 10.4. Deckle Windows v0.4
- 10.5. Deckle Windows v0.5
- 10.6. Deckle Windows v0.6
- 10.7. GrainLayer v0.8
- 10.8. GrainLayer v0.9
- 10.9. GrainLayer v1.0
- 10.10. GrainLayer v1.0.1
- 10.11. GrainLayer v1.1
- 10.12. GrainLayer v1.2
- 10.13. GrainLayer v1.2.1
- 10.14. GrainLayer v1.2.2
- 10.15. GrainLayer v1.2.3
- 10.16. GrainLayer v1.2.4
- 10.17. GrainLayer v1.2.5
- 10.18. GrainLayer v1.2.6
- 10.19. GrainLayer v1.3
- 10.20. GrainLayer v1.4
- 10.21. GrainLayer v1.4.1
- 11. Appendix B --- Historical Version Map
- 12. Appendix C --- Credits and Inspiration
 - 12.1. Paperman.cc
 - 12.2. Deckle
- 13. Appendix D --- Final Notes for the Curious Programmer

0.1. Building a Paper Layer, One Win32 Version at a Time

Project: GrainLayer\ **Historical prototype name:** Deckle Windows\ **Language:** C++\ **Platform:** Microsoft Windows / Win32\
Current documented release: v1.4.1

0.2. Preface

This manual is deliberately written in the style of the old programming books that treated source code as something to be read at a desk, line by line.

GrainLayer is a particularly good subject for that treatment because the finished program is small enough to understand, yet it touches a surprisingly large part of native Windows programming: window classes, message loops, layered windows, alpha compositing, device-independent bitmaps, shell notification icons, menus, owner-drawn controls, mouse capture, message filters, registry settings, application-data storage, resources, hotkeys and Visual Studio project files.

The program did not begin as GrainLayer. The first preserved source snapshot is **Deckle Windows v0.1**. The name was later changed to GrainLayer so that the Windows project would clearly be an independent project rather than an official Windows port of Deckle.

The original idea came from **Deckle by Akshat Katiyar**, a macOS application that applies a subtle paper-like texture over the desktop. GrainLayer is an independent implementation inspired by that concept.

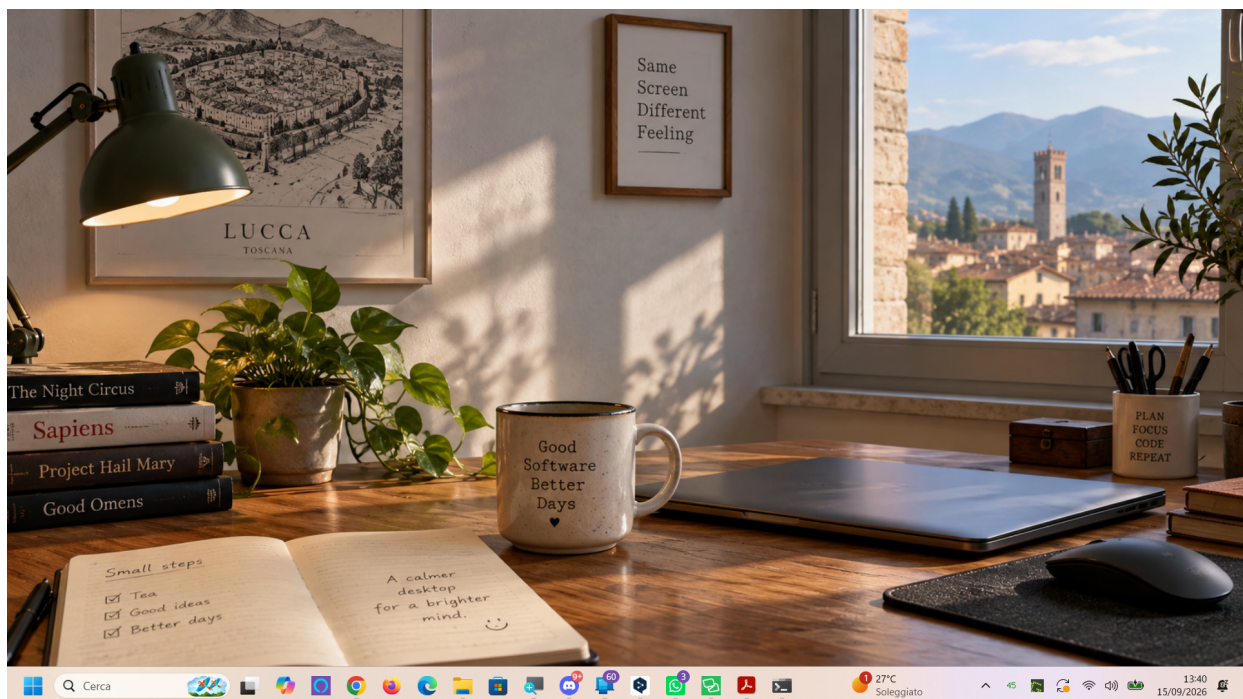
This book documents the development as an engineering story rather than pretending that the final source appeared fully formed. Bugs, wrong assumptions and awkward intermediate designs are included because they are often the most useful part of a programming history.

0.3. Current Build: Visual Reference

The following screenshots document the finished v1.4.1 application as it looks on Windows.

0.3.1. The same desktop, without and with the overlay

The first two images are deliberately the same desktop captured at the same resolution. The only meaningful difference is whether GrainLayer is enabled.



Windows desktop without GrainLayer

Figure 1 — The desktop with GrainLayer disabled.



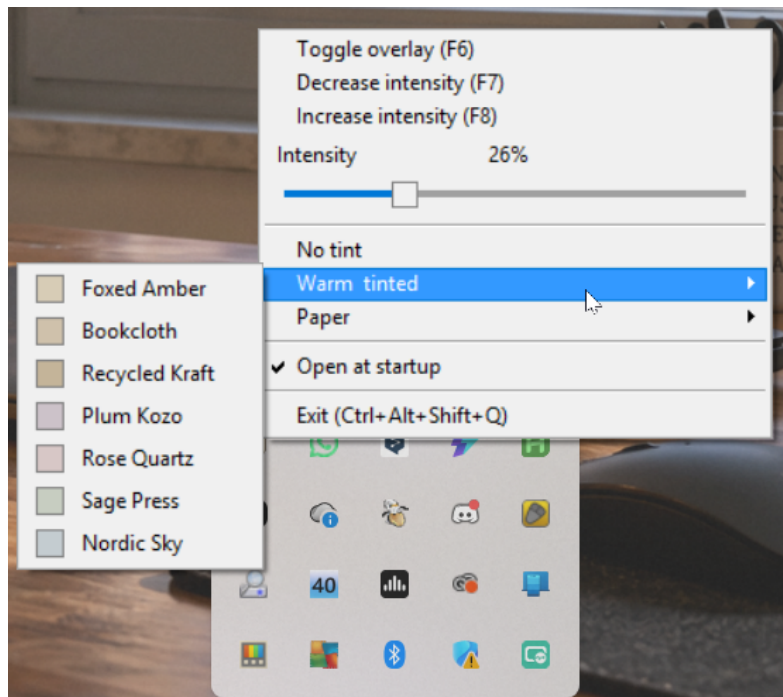
Windows desktop with GrainLayer

Figure 2 — The same desktop with GrainLayer enabled. The overlay is

deliberately subtle; its purpose is to change the surface character of the display rather than to apply a conspicuous visual filter.*

0.3.2. The tray interface

The third image shows the main user interface. GrainLayer has no ordinary application window: the system-tray menu is the control surface.



GrainLayer tray menu

Figure 3 — The tray menu with the intensity control and Warm tinted

presets visible. The screenshot shows an intensity of 26%.*

These images are useful when reading the later chapters because they provide a visual reference for the controls described in the source: F6 toggles the overlay, F7 and F8 adjust intensity, and the owner-drawn slider provides direct mouse control.

0.4. How to Read This Manual

Each chapter has three layers:

****What changed**** --- the purpose of the version.

****Why it works**** --- the relevant Windows/C++ mechanism.

****Source listing**** --- the complete preserved ``main.cpp`` for that version.

The complete listings are deliberately retained in the appendices. They are historical snapshots, not recommendations to copy every intermediate implementation into a new project.

Where a historical source snapshot is unavailable, that fact is stated rather than reconstructed from memory.

1. Part I --- The First Window

1.1. Chapter 0.1 --- Deckle Windows v0.1

The project began as a Windows experiment called Deckle. The objective was deliberately modest: put a paper-like visual layer over the Windows desktop and make it controllable with a few hotkeys.

The first implementation already established an important architectural idea: the application did not need a conventional visible main window. A hidden Win32 window could own the message loop and hotkeys, while the desktop effect lived in a separate visual surface.

The first version used `SetLayeredWindowAttributes` with a uniform window alpha. This was enough to make a transparent-looking overlay, but it was not yet the per-pixel composition model that the later versions would require.

Programming lesson: start with the smallest observable result. At this stage the question was not "How do we build a polished utility?" but "Can a native Win32 process create the visual effect at all?" `### Chapter 0.2 --- Deckle Windows v0.2`

Version 0.2 continued the same architecture and refined the overlay. The important thing to notice is that the application was still solving the problem in layers: first create a window, then make it topmost, then make it transparent/click-through, then add control.

The code remained intentionally small enough that the Windows message loop and window procedure could still be understood as a single unit.

Programming lesson: when learning Win32, it is useful to keep the event loop visible rather than immediately hiding it behind abstractions. `## Chapter 0.3 --- Deckle Windows v0.3`

Version 0.3 was the first major technical turning point.

The uniform-alpha approach was not sufficient for the visual effect. The overlay needed actual pixel data with an alpha channel. The program therefore switched to a 32-bit DIB section and `UpdateLayeredWindow`.

The important detail is premultiplied alpha. `UpdateLayeredWindow` with `AC_SRC_ALPHA` expects the RGB components to have been multiplied by the alpha value. The source image is therefore prepared in memory before Windows composites it.

This is the version where the classic "why is my transparent window black?" lesson appears: transparency is not one feature but a chain of compatible bitmap format, alpha representation, and window-composition settings.

Programming lesson: graphical bugs often come from mismatched representations rather than from a single wrong line. `## Chapter 0.4 --- Deckle Windows v0.4`

Version 0.4 preserved the new per-pixel renderer and added another small piece of interaction: Escape could be registered as an additional hotkey.

This is a good example of the project becoming an actual utility rather than a rendering experiment. The visual layer and the command layer were now distinct concerns. `## Chapter 0.5 --- Deckle Windows v0.5`

Version 0.5 introduced the system tray. `Shell_NotifyIconW` gave the otherwise invisible utility a persistent user interface, and the hotkeys were complemented by menu commands.

The intensity value was centralized in `ChangeOpacity`, so both keyboard and menu actions could modify the same state.

A dedicated emergency exit combination, `Ctrl+Alt+Shift+Q`, also appeared here.

Programming lesson: once an application has no ordinary main window, the notification area becomes its natural control surface. ## Chapter 0.6 --- Deckle Windows v0.6

Version 0.6 was primarily a stabilization pass over the tray-enabled application. The rendering architecture remained unchanged, while the shell integration and dependencies were made explicit.

This stage is important because it demonstrates that "nothing visually new" can still be a meaningful release: keeping a working architecture intact while removing rough edges is part of development. ## Chapter 0.8 --- GrainLayer v0.8

The project was renamed GrainLayer and moved into a more feature-oriented stage. The core renderer remained the stable 32-bit DIB plus UpdateLayeredWindow implementation.

The menu grew into a more complete control panel, with overlay state, intensity controls, tint/paper presets and the keyboard shortcuts that would remain part of the application's identity.

Programming lesson: a stable rendering core should become a fixed point. Later features should call into it rather than repeatedly replacing it. ## Chapter 0.9 --- GrainLayer v0.9

Version 0.9 concentrated on stabilization before the 1.x line. The architecture had converged: hidden owner window, tray icon, layered desktop overlay, shared state, hotkeys and menu commands.

The source is already considerably larger than the original experiment, which is a useful reminder that a small desktop utility can accumulate complexity quickly once it has to behave like a real Windows application. ## Chapter 1.0 --- GrainLayer v1.0

Version 1.0 marked the first stable project structure. The Visual Studio solution and project metadata were made conventional, with explicit x64 configurations and the v143 toolset.

This is less glamorous than the overlay itself, but it is crucial for reproducibility: source code is only useful as a project if another developer can open the solution, select a known configuration and build it. ## Chapter 1.0.1 --- GrainLayer v1.0.1

Version 1.0.1 is retained as a source snapshot because it represents the bridge into the graphical-control work. The core renderer and tray architecture are still recognizable, while the code is prepared for the next user-interface milestone. ## Chapter 1.1 --- GrainLayer v1.1

Version 1.1 introduced the first graphical intensity control. The application no longer relied only on discrete keyboard/menu commands: intensity became something the user could manipulate as a continuous value.

This is where UI state and rendering state began to interact more closely. A control that displays intensity must always be derived from the same value that controls the overlay. ## Chapter 1.2 --- GrainLayer v1.2

Version 1.2 moved the slider directly into the tray menu as an owner-drawn menu item. This was a deliberately Win32-native solution rather than a separate dialog.

Owner drawing means Windows asks the application to paint the item itself. The program therefore had to understand both the menu's geometry and the current intensity value.

The first implementation also exposed an easy-to-make Win32 mistake: `GetMenuItemRect` expects a menu position, not an arbitrary command identifier. ## Chapter 1.2.1 --- GrainLayer v1.2.1

Version 1.2.1 continued the owner-drawn slider implementation and addressed small type/interaction issues. The important design remained unchanged: the slider lived inside the popup menu and was painted from application state. ## Chapter 1.2.2 --- GrainLayer v1.2.2

Version 1.2.2 added the machinery required for true click-and-drag interaction. The menu's modal tracking loop meant ordinary window mouse messages were not enough, so a `WM_MSGFILTER` hook was introduced.

During an active drag, the application captured the mouse to the popup menu window (`#32768`) and updated the intensity from `WM_MOUSEMOVE`.

This version also contained an accidental duplicated `{` during editing, producing a cascade of compiler errors. That failure is worth preserving in a programming manual because C++ diagnostics often point downstream from the real syntax error. ## Chapter 1.2.3 --- GrainLayer v1.2.3

Version 1.2.3 was a repair of the previous source-level mistake. The drag architecture itself remained intact.

The lesson is procedural: after a cascading C++ error, find the earliest structural error first. Later errors may simply be the compiler losing synchronization with the source. ## Chapter 1.2.4 --- GrainLayer v1.2.4

Version 1.2.4 fixed the slider hit-testing bug properly.

The crucial discovery was that menu command IDs and menu positions are different concepts. `GetMenuItemRect` uses the zero-based position of the item. The code therefore iterates over `GetMenuItemCount`, reads each item's `MIIM_ID`, finds

IDM_INTENSITY_SLIDER, and only then asks Windows for its rectangle.

This is a classic Win32 lesson: many APIs use either an identifier or a positional index, and the two are not interchangeable. ## Chapter 1.2.5 --- GrainLayer v1.2.5

Version 1.2.5 made click-and-drag live. A left click establishes the slider position; while dragging, WM_MOUSEMOVE updates the value continuously; releasing the mouse ends the drag.

The popup menu's special window class (#32768) is part of the solution because the menu itself owns the interaction while it is being tracked.

1.2. Chapter 1.2.6 --- GrainLayer v1.2.6

Version 1.2.6 refined the interaction model rather than adding another feature.

Hovering over the slider no longer changed anything. A click explicitly positions the slider, and a click followed by movement performs a drag. This matches familiar Windows control behaviour and avoids surprising state changes simply because the pointer passed over the control.

Programming lesson: interaction semantics are part of correctness, not merely polish. ## Chapter 1.3 --- GrainLayer v1.3

Version 1.3 added the custom GrainLayer icon as a resource. This was a small change technically, but it completed the transition from prototype to recognizable desktop utility.

The icon is compiled through the .rc resource file and embedded in the executable rather than loaded from an external file at runtime. ## Chapter 1.4 --- GrainLayer v1.4

Version 1.4 added persistent settings and optional Windows startup.

Settings are stored under the user's roaming application-data directory in settings.ini. The program saves overlay state, intensity and tint with WritePrivateProfileStringW, and restores them at startup with GetPrivateProfileStringW.

Startup is implemented through the current user's Software\Microsoft\Windows\CurrentVersion\Run key. Because it is HKCU rather than HKLM, administrator privileges are not required.

The menu wording was also changed to Toggle overlay (F6), which describes the actual action more accurately. ## Chapter 1.4.1 --- GrainLayer v1.4.1

Version 1.4.1 is a small corrective release.

The keyboard controls were made logically consistent with the visual direction of the slider: **F7 decreases intensity and F8 increases it**, so the left/decrease action and right/increase action follow the convention used by volume and brightness controls.

The slider is also invalidated and redrawn when F7 or F8 changes intensity while the menu is open. The displayed position therefore remains synchronized with the actual overlay state.

No change was made to the stable layered-window renderer or to the click-and-drag interaction. # Part II --- The Architecture

1.3. 1. The Invisible Owner Window

GrainLayer is fundamentally a message-driven Win32 program.

There is no ordinary application window that the user works inside. Instead, a hidden or unobtrusive owner window provides:

- the message loop;
- hotkey registration;
- tray-icon ownership;
- menu commands;
- application lifetime management.

The desktop effect is provided by a separate layered window.

This division is useful because the visual surface and the control surface have different jobs.

1.4. 2. The Layered Window

The visual overlay uses the extended window style WS_EX_LAYERED.

The critical rendering path is:

determine the virtual desktop dimensions;

create a 32-bit DIB section;

obtain direct access to its pixel buffer;

generate the grain/tint pixels;

premultiply RGB channels by alpha;

present the bitmap through `UpdateLayeredWindow``;

keep the window click-through so normal desktop interaction

continues underneath.

The use of the **virtual screen** rather than a single monitor is what lets GrainLayer cover a multi-monitor desktop.

1.5. 3. Premultiplied Alpha

Suppose a source pixel has channels:

R, G, B, A

For a premultiplied-alpha bitmap, the stored colour channels become approximately:

```
R' = R * A / 255
G' = G * A / 255
B' = B * A / 255
```

The alpha channel remains A.

This matters because the Windows layered-window compositor expects the RGB values to already reflect the alpha channel when `AC_SRC_ALPHA` is used.

1.6. 4. Click-Through Behaviour

The overlay must be visible without stealing input from the applications underneath it.

The window is therefore created with the appropriate layered/transparent extended styles and does not act as a normal interactive desktop window.

This is a recurring theme in Win32 programming: **visibility and hit-testing are separate concerns.**

2. Part III --- The Tray Application

2.1. 1. Why a Tray Icon?

A utility that has no normal window still needs a user interface.

The notification area provides exactly that:

```
Shell_NotifyIconW(NIM_ADD, &g_nid);
```

The tray icon remains visible while the overlay itself covers the desktop.

The menu becomes the application's compact control panel.

2.2. 2. Command IDs

The menu is built around integer command identifiers.

A command ID is not a menu position.

This distinction became particularly important with the slider. A command might have:

```
IDM_INTENSITY_SLIDER = 1006
```

while occupying, for example, position 3 in the popup menu.

APIs that ask for a **position** must receive 3, not 1006.

2.3. 3. Hotkeys

The application registers:

F6	Toggle overlay
F7	Decrease intensity
F8	Increase intensity
Ctrl+Alt+Shift+Q	Exit

The hotkey IDs are internal identifiers; they are not the same thing as the physical key codes.

3. Part IV --- Building a Slider Inside a Win32 Menu

This is the most instructive part of the project because the control is not a conventional child window.

3.1. 1. Owner-Drawn Menu Items

A menu item can be marked owner-drawn. Windows then sends drawing information to the application, allowing the program to paint a custom control.

For GrainLayer the item represents:

Intensity: 

The knob position is calculated from the current intensity.

3.2. 2. Finding the Item Rectangle

The robust procedure is:

call ``GetMenuItemCount``;

iterate from position zero;

call ``GetMenuItemInfoW`` with ``MIIM_ID``;

compare the returned command ID with ``IDM_INTENSITY_SLIDER``;

call ``GetMenuItemRect`` using the discovered `**position**`.

This is a small piece of code with a large lesson behind it: always check what a Win32 parameter actually means.

3.3. 3. Click-to-Position

A left mouse click within the slider rectangle is converted into an intensity value.

Conceptually:

```
mouse X
  ↓
subtract left edge
  ↓
divide by usable slider width
  ↓
clamp to 0..100
  ↓
update intensity
  ↓
redraw menu
```

3.4. 4. Dragging

A drag needs continuous mouse movement.

While the popup menu is active, the normal application window is not receiving all mouse messages in the usual way. GrainLayer therefore uses a `WH_MSGFILTER` hook to observe messages being processed by the menu's modal loop.

When a drag begins:

```
left button down
  ↓
set dragging = true
  ↓
capture the popup menu window
  ↓
WM_MOUSEMOVE → update intensity
  ↓
left button up
  ↓
```

```
release capture
  ↓
set dragging = false
```

3.5. 5. Why Hover Does Nothing

An earlier interaction model changed the value simply because the pointer moved over the slider.

That was discarded.

The final behaviour is deliberately explicit:

- **hover:** nothing happens;
- **click:** position the slider;
- **click + drag:** update continuously;
- **release:** finish the drag.

This is a useful example of a small UI decision being treated as a correctness issue.

4. Part V --- Persistent State

Version 1.4 introduced persistence.

4.1. 1. Where the Settings Live

The program uses the user's roaming application-data directory:

```
%APPDATA%\GrainLayer\settings.ini
```

The directory is created if necessary.

4.2. 2. What Is Stored

The persistent state includes:

- whether the overlay is enabled;
- intensity;
- selected tint.

4.3. 3. INI Storage

The Win32 profile APIs are sufficient for such a small amount of state:

```
WritePrivateProfileStringW(...)
GetPrivateProfileStringW(...)
```

This is not intended as a general configuration database. It is simply an appropriate lightweight mechanism for three small settings.

4.4. 4. Windows Startup

Optional startup uses:

```
HKEY_CURRENT_USER
  Software
    Microsoft
      Windows
        CurrentVersion
          Run
```

Using HKEY_CURRENT_USER means the setting belongs to the logged-in user and does not require administrative privileges.

5. Part VI --- Resources and the Executable

The icon is compiled into the program through a Windows resource script.

The relevant chain is:

```
GrainLayer.ico
↓
GrainLayer.rc
↓
resource.h
↓
Visual Studio resource compilation
↓
GrainLayer.exe
```

This means the finished application does not need to locate its icon beside the executable.

6. Part VII --- The Visual Studio Project

The project is deliberately conventional:

```
GrainLayer.sln
GrainLayer.vcxproj
src/
  main.cpp
resources/
  GrainLayer.rc
  GrainLayer.ico
resource.h
docs/
```

The build configuration uses:

- Visual Studio 2022;
- v143 toolset;
- Windows SDK;
- C++20;
- x64;
- Unicode.

The final project also explicitly supplies the include path needed for the root-level `resource.h` and links the Windows libraries required by the application.

7. Part VIII --- What the Bugs Taught

7.1. The Black Overlay

Symptom: the overlay appeared opaque or black.

Cause: the rendering path did not correctly provide the per-pixel alpha information expected by the layered-window compositor.

Fix: 32-bit DIB + premultiplied alpha + `UpdateLayeredWindow`.

Lesson: bitmap format, alpha representation and composition API must agree.

7.2. Missing Shell Declarations

Symptom: tray-related symbols were unavailable.

Fix: include the appropriate Shell API header and link the Shell library.

Lesson: a Win32 API feature generally has both a declaration side and a linker side.

7.3. Menu Position vs. Command ID

Symptom: the slider rectangle could not be found.

Cause: a command ID was passed where a zero-based menu position was required.

Fix: enumerate menu positions and resolve the command ID first.

Lesson: identifiers and indexes are different data types even when both are integers.

7.4. Cascading C++ Errors

Symptom: a small editing mistake produced a long series of syntax errors.

Cause: one unmatched or duplicated brace changed the compiler's interpretation of the rest of the function.

Lesson: in a cascade, inspect the first structural error before fixing the later diagnostics.

7.5. Mouse Dragging Inside a Popup Menu

Symptom: ordinary mouse handling did not provide the required continuous drag behaviour.

Fix: message filtering plus mouse capture on the menu window.

Lesson: menus are special modal UI surfaces in Win32; they do not behave exactly like ordinary application windows.

7.6. Slider Synchronization

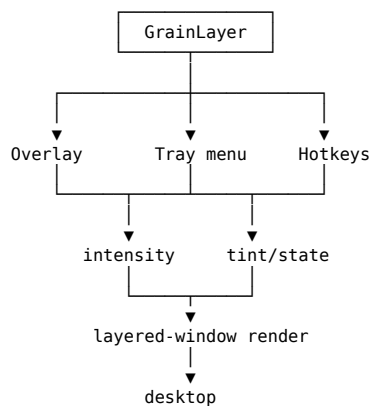
Symptom: keyboard intensity changes altered the overlay but left the visible slider at its old position.

Fix: invalidate/redraw the owner-drawn slider after F7/F8 changes.

Lesson: every visual representation of mutable state must have an explicit synchronization path.

8. Part IX --- Final Behaviour

At v1.4.1 the program can be understood as a small state machine:



The important point is that there is one logical state. The tray menu, keyboard commands and renderer all operate on that state rather than maintaining independent copies.

9. Part X --- A Short Tour of the Final Source

When reading `src/main.cpp`, a useful order is:

****Constants and global state**** --- learn what the program remembers.

****Rendering functions**** --- understand how the pixels are produced.

****Settings functions**** --- understand persistence.

****Registry functions**** --- understand startup.

****Menu construction**** --- understand the user interface.

****Slider geometry and drawing**** --- understand owner-drawn controls.

****Message filter**** --- understand dragging.

****Window procedure**** --- understand how commands enter the program.

****wWinMain**** --- understand startup, hotkeys, tray registration

and the message loop.

This order follows the data flow rather than the physical order of the source file.

10. Appendix A --- Complete Historical Source Listings

The following listings are preserved source snapshots from the development archive. They are included for study and comparison.

10.1. Deckle Windows v0.1

```
#define UNICODE
#define _UNICODE
#define WIN32_LEAN_AND_MEAN
#define NOMINMAX

#include <windows.h>
#include <d2d1.h>
#include <dwrite.h>
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <random>
#include <vector>

#pragma comment(lib, "d2d1.lib")
#pragma comment(lib, "dwrite.lib")

static ID2D1Factory* g_factory = nullptr;
static ID2D1HwndRenderTarget* g_target = nullptr;
static ID2D1Bitmap* g_bitmap = nullptr;
static ID2D1SolidColorBrush* g_brush = nullptr;
static HWND g_hwnd = nullptr;
static BYTE g_alpha = 34; // roughly 13%
static bool g_enabled = true;
static int g_virtualW = 0, g_virtualH = 0;

struct Pixel { BYTE b,g,r,a; };

static void SafeRelease(IUnknown*& p) { if (p) { p->Release(); p = nullptr; } }

static uint32_t Hash(uint32_t x) {
    x ^= x >> 16; x *= 0x7feb352dU;
    x ^= x >> 15; x *= 0x846ca68bU;
    x ^= x >> 16; return x;
}

static void BuildTexture() {
    SafeRelease(reinterpret_cast<IUnknown*>(g_bitmap));

    const int W = 1024, H = 1024;
    std::vector<Pixel> px(W * H);

    for (int y = 0; y < H; ++y) {
        for (int x = 0; x < W; ++x) {
            uint32_t h = Hash((uint32_t)x * 374761393u ^ (uint32_t)y * 668265263u);
            float fine = ((h & 0xffffu) / 65535.0f) - 0.5f;

            // A few weak, elongated "fibres", plus fine grain.
            float fibre = 0.0f;
            for (int k = 0; k < 3; ++k) {
                int yy = (y + k * 311) & 1023;
                float d = std::abs(std::sin((x + k * 137) * 0.018f) * 9.0f);
                fibre += (d < 1.0f && ((yy + x * 3) % 47) == 0) ? 0.18f : 0.0f;
            }

            float v = 0.5f + fine * 0.10f + fibre;
            v = std::clamp(v, 0.25f, 0.75f);

            BYTE c = (BYTE)(v * 255.0f);
            px[y * W + x] = { c, c, c, 255 };
        }
    }

    D2D1_BITMAP_PROPERTIES props = {};
    props.pixelFormat.format = DXGI_FORMAT_B8G8R8A8_UNORM;
    props.pixelFormat.alphaMode = D2D1_ALPHA_MODE_IGNORE;
    props.dpiX = props.dpiY = 96.0f;

    HRESULT hr = g_target->CreateBitmap(
        D2D1::SizeU(W, H), px.data(), W * sizeof(Pixel), &props, &g_bitmap);

    if (FAILED(hr)) g_bitmap = nullptr;
}

static void ResizeTarget() {
    if (!g_target) return;
    RECT rc{};
    GetClientRect(g_hwnd, &rc);
    g_target->Resize(D2D1::SizeU(
        std::max(1L, rc.right - rc.left),
        std::max(1L, rc.bottom - rc.top)));
}
```

```

static void Draw() {
    if (!g_target) return;
    RECT rc{};
    GetClientRect(g_hwnd, &rc);

    g_target->BeginDraw();
    g_target->Clear(D2D1::ColorF(0, 0.0f));

    if (g_enabled && g_bitmap) {
        const float a = g_alpha / 255.0f;
        g_target->DrawBitmap(
            g_bitmap,
            D2D1::RectF(0, 0,
                (FLOAT)(rc.right - rc.left),
                (FLOAT)(rc.bottom - rc.top)),
            a,
            D2D1_INTERPOLATION_MODE_HIGH_QUALITY_CUBIC);
    }

    HRESULT hr = g_target->EndDraw();
    if (hr == D2DERR_RECREATE_TARGET) {
        SafeRelease(reinterpret_cast<IUnknown*>(g_bitmap));
        SafeRelease(reinterpret_cast<IUnknown*>(g_brush));
        SafeRelease(reinterpret_cast<IUnknown*>(g_target));
        InvalidateRect(g_hwnd, nullptr, FALSE);
    }
}

static HRESULT CreateD2D() {
    HRESULT hr = D2D1CreateFactory(D2D1_FACTORY_TYPE_SINGLE_THREADED, &g_factory);
    if (FAILED(hr)) return hr;

    RECT rc{};
    GetClientRect(g_hwnd, &rc);
    hr = g_factory->CreateHwndRenderTarget(
        D2D1::RenderTargetProperties(
            D2D1_RENDER_TARGET_TYPE_DEFAULT,
            D2D1::PixelFormat(DXGI_FORMAT_B8G8R8A8_UNORM,
                D2D1_ALPHA_MODE_PREMULTIPLIED)),
        D2D1::HwndRenderTargetProperties(
            g_hwnd,
            D2D1::SizeU(std::max(1L, rc.right - rc.left),
                std::max(1L, rc.bottom - rc.top)),
            D2D1_PRESENT_OPTIONS_RETAIN_CONTENTS),
        &g_target);

    if (SUCCEEDED(hr)) BuildTexture();
    return hr;
}

static BOOL CALLBACK MonitorEnum(HMONITOR, HDC, LPRECT r, LPARAM) {
    g_virtualW = std::max(g_virtualW, r->right);
    g_virtualH = std::max(g_virtualH, r->bottom);
    return TRUE;
}

static void GetVirtualDesktop() {
    g_virtualW = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_virtualH = GetSystemMetrics(SM_YVIRTUALSCREEN);
    int right = g_virtualW + GetSystemMetrics(SM_CXVIRTUALSCREEN);
    int bottom = g_virtualH + GetSystemMetrics(SM_CYVIRTUALSCREEN);
    (void)right; (void)bottom;
}

static LRESULT CALLBACK WndProc(HWND h, UINT msg, WPARAM w, LPARAM l) {
    switch (msg) {
        case WM_NCHITTEST:
            return HTTRANSPARENT;

        case WM_ERASEBKGD:
            return 1;

        case WM_SIZE:
            ResizeTarget();
            return 0;

        case WM_DISPLAYCHANGE:
            GetVirtualDesktop();
            SetWindowPos(h, HWND_TOPMOST,
                GetSystemMetrics(SM_XVIRTUALSCREEN),
                GetSystemMetrics(SM_YVIRTUALSCREEN),
                GetSystemMetrics(SM_CXVIRTUALSCREEN),
                GetSystemMetrics(SM_CYVIRTUALSCREEN),
                SWP_NOACTIVATE | SWP_SHOWWINDOW);
            ResizeTarget();
            InvalidateRect(h, nullptr, FALSE);
            return 0;

        case WM_PAINT: {
            PAINTSTRUCT ps{};
            BeginPaint(h, &ps);
            Draw();
            EndPaint(h, &ps);
            return 0;
        }

        case WM_HOTKEY:
            if (w == 1) { g_enabled = !g_enabled; InvalidateRect(h, nullptr, FALSE); }
    }
}

```

```

        else if (w == 2) { g_alpha = (BYTE)std::min(100, (int)g_alpha + 5); InvalidateRect(h, nullptr, FALSE); }
        else if (w == 3) { g_alpha = (BYTE)std::max(0, (int)g_alpha - 5); InvalidateRect(h, nullptr, FALSE); }
        return 0;

    case WM_KEYDOWN:
        if (w == VK_ESCAPE) PostQuitMessage(0);
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(h, msg, w, l);
}

int WINAPI wWinMain(HINSTANCE inst, HINSTANCE, PWSTR, int) {
    GetVirtualDesktop();

    WNDCLASSEXW wc{ sizeof(wc) };
    wc.hInstance = inst;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"DeckleWindowsOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
    RegisterClassExW(&wc);

    const int x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    const int y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    const int w = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    const int h = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    DWORD ex = WS_EX_LAYERED | WS_EX_TRANSPARENT | WS_EX_NOACTIVATE |
               WS_EX_TOOLWINDOW | WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        ex, wc.lpszClassName, L"DeckleWindows",
        WS_POPUP, x, y, w, h, nullptr, nullptr, inst, nullptr);

    if (!g_hwnd) return 1;

    // The window is visually transparent except for what Direct2D draws.
    // A layered window is used so the untouched pixels remain transparent.
    SetLayeredWindowAttributes(g_hwnd, 0, 255, LWA_ALPHA);

    if (FAILED(CreateD2D())) return 2;

    RegisterHotKey(g_hwnd, 1, MOD_NOREPEAT, VK_F6);
    RegisterHotKey(g_hwnd, 2, MOD_NOREPEAT, VK_F7);
    RegisterHotKey(g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    UpdateWindow(g_hwnd);

    MSG msg{};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);

    SafeRelease(reinterpret_cast<IUnknown*>(g_bitmap));
    SafeRelease(reinterpret_cast<IUnknown*>(g_brush));
    SafeRelease(reinterpret_cast<IUnknown*>(g_target));
    SafeRelease(reinterpret_cast<IUnknown*>(g_factory));
    return 0;
}

```

10.2. Deckle Windows v0.2

```

#include <windows.h>
#include <d2d1.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "d2d1.lib")

static ID2D1Factory* g_factory = nullptr;
static ID2D1HwndRenderTarget* g_target = nullptr;
static ID2D1Bitmap* g_bitmap = nullptr;
static HWND g_hwnd = nullptr;

static BYTE g_alpha = 34;
static bool g_enabled = true;

struct Pixel {
    BYTE b, g, r, a;
};

static void SafeRelease(IUnknown*& p)
{
    if (p) {
        p->Release();
        p = nullptr;
    }
}

```

```

}

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void BuildTexture()
{
    SafeRelease(reinterpret_cast<IUnknown*>(g_bitmap));

    const UINT W = 1024;
    const UINT H = 1024;
    std::vector<Pixel> pixels(W * H);

    for (UINT y = 0; y < H; ++y) {
        for (UINT x = 0; x < W; ++x) {
            const uint32_t h =
                Hash(x * 374761393u ^ y * 668265263u);

            // Fine paper grain.
            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.5f + noise * 0.10f;
            if (value < 0.25f) value = 0.25f;
            if (value > 0.75f) value = 0.75f;

            const BYTE c = static_cast<BYTE>(value * 255.0f);
            pixels[y * W + x] = { c, c, c, 255 };
        }
    }

    D2D1_BITMAP_PROPERTIES props = {};
    props.pixelFormat.format = DXGI_FORMAT_B8G8R8A8_UNORM;
    props.pixelFormat.alphaMode = D2D1_ALPHA_MODE_IGNORE;
    props.dpiX = 96.0f;
    props.dpiY = 96.0f;

    HRESULT hr = g_target->CreateBitmap(
        D2D1::SizeU(W, H),
        pixels.data(),
        W * sizeof(Pixel),
        &props,
        &g_bitmap);

    if (FAILED(hr))
        g_bitmap = nullptr;
}

static void ResizeTarget()
{
    if (!g_target)
        return;

    RECT rc = {};
    GetClientRect(g_hwnd, &rc);

    const UINT width =
        rc.right > rc.left ? static_cast<UINT>(rc.right - rc.left) : 1;
    const UINT height =
        rc.bottom > rc.top ? static_cast<UINT>(rc.bottom - rc.top) : 1;

    g_target->Resize(D2D1::SizeU(width, height));
}

static void Draw()
{
    if (!g_target)
        return;

    RECT rc = {};
    GetClientRect(g_hwnd, &rc);

    const float width = static_cast<float>(rc.right - rc.left);
    const float height = static_cast<float>(rc.bottom - rc.top);

    g_target->BeginDraw();

    // Transparent background.
    g_target->Clear(D2D1::ColorF(0.0f, 0.0f));

    if (g_enabled && g_bitmap) {
        const float opacity = static_cast<float>(g_alpha) / 255.0f;

        // LINEAR is supported by older Windows SDKs too.
        g_target->DrawBitmap(
            g_bitmap,
            D2D1::RectF(0.0f, 0.0f, width, height),
            opacity,
            D2D1_BITMAP_INTERPOLATION_MODE_LINEAR);
    }
}

```

```

    HRESULT hr = g_target->EndDraw();

    if (hr == D2DERR_RECREATE_TARGET) {
        SafeRelease(reinterpret_cast<IUnknown*>(g_bitmap));
        SafeRelease(reinterpret_cast<IUnknown*>(g_target));
        InvalidateRect(g_hwnd, nullptr, FALSE);
    }
}

static HRESULT CreateD2D()
{
    HRESULT hr =
        D2D1CreateFactory(D2D1_FACTORY_TYPE_SINGLE_THREADED, &g_factory);

    if (FAILED(hr))
        return hr;

    RECT rc = {};
    GetClientRect(g_hwnd, &rc);

    const UINT width =
        rc.right > rc.left ? static_cast<UINT>(rc.right - rc.left) : 1;
    const UINT height =
        rc.bottom > rc.top ? static_cast<UINT>(rc.bottom - rc.top) : 1;

    hr = g_factory->CreateHwndRenderTarget(
        D2D1::RenderTargetProperties(
            D2D1_RENDER_TARGET_TYPE_DEFAULT,
            D2D1::PixelFormat(
                DXGI_FORMAT_B8G8R8A8_UNORM,
                D2D1_ALPHA_MODE_PREMULTIPLIED)),
        D2D1::HwndRenderTargetProperties(
            g_hwnd,
            D2D1::SizeU(width, height),
            D2D1_PRESENT_OPTIONS_NONE),
        &g_target);

    if (SUCCEEDED(hr))
        BuildTexture();

    return hr;
}

static LRESULT CALLBACK WndProc(
    HWND h, UINT msg, WPARAM wParam, LPARAM lParam)
{
    switch (msg) {
    case WM_NCHITTEST:
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_ERASEBKGD:
        return 1;

    case WM_SIZE:
        ResizeTarget();
        return 0;

    case WM_DISPLAYCHANGE:
        SetWindowPos(
            h,
            HWND_TOPMOST,
            GetSystemMetrics(SM_XVIRTUALSCREEN),
            GetSystemMetrics(SM_YVIRTUALSCREEN),
            GetSystemMetrics(SM_CXVIRTUALSCREEN),
            GetSystemMetrics(SM_CYVIRTUALSCREEN),
            SWP_NOACTIVATE | SWP_SHOWWINDOW);
        ResizeTarget();
        InvalidateRect(h, nullptr, FALSE);
        return 0;

    case WM_PAINT:
    {
        PAINTSTRUCT ps = {};
        BeginPaint(h, &ps);
        Draw();
        EndPaint(h, &ps);
        return 0;
    }

    case WM_HOTKEY:
        if (wParam == 1) {
            g_enabled = !g_enabled;
            InvalidateRect(h, nullptr, FALSE);
        }
        else if (wParam == 2) {
            g_alpha = static_cast<BYTE>(
                (g_alpha + 5 <= 100) ? g_alpha + 5 : 100);
            InvalidateRect(h, nullptr, FALSE);
        }
        else if (wParam == 3) {
            g_alpha = static_cast<BYTE>(
                (g_alpha >= 5) ? g_alpha - 5 : 0);
            InvalidateRect(h, nullptr, FALSE);
        }
        return 0;
    }
}

```

```

        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;
        }

        return DefWindowProc(h, msg, wParam, lParam);
    }

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"DeckleWindowsOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    const int x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    const int y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    const int width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    const int height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"DeckleWindows",
        WS_POPUP,
        x, y, width, height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    SetLayeredWindowAttributes(g_hwnd, 0, 255, LWA_ALPHA);

    if (FAILED(CreateD2D()))
        return 3;

    RegisterHotKey(g_hwnd, 1, MOD_NOREPEAT, VK_F6);
    RegisterHotKey(g_hwnd, 2, MOD_NOREPEAT, VK_F7);
    RegisterHotKey(g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    UpdateWindow(g_hwnd);

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);

    SafeRelease(reinterpret_cast<IUnknown*>(g_bitmap));
    SafeRelease(reinterpret_cast<IUnknown*>(g_target));
    SafeRelease(reinterpret_cast<IUnknown*>(g_factory));

    return 0;
}

```

10.3. Deckle Windows v0.3

```

#include <windows.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")

static HWND g_hwnd = nullptr;
static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;

```

```

static int g_height = 0;

static BYTE g_alpha = 34; // ~13% opacity
static bool g_enabled = true;

struct Pixel {
    BYTE b, g, r, a;
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void DestroyLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    DestroyLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height; // top-down
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        DestroyLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);

    return true;
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);

    // Generate a 1024x1024 procedural paper tile and repeat it over
    // the virtual desktop. The overlay itself is the only thing rendered;
    // the underlying screen is never captured.
    constexpr int TILE = 1024;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            if (!g_enabled || g_alpha == 0) {
                pixels[y * g_width + x] = { 0, 0, 0, 0 };
            }
        }
    }
}

```

```

        continue;
    }

    const int tx = x & (TILE - 1);
    const int ty = y & (TILE - 1);

    uint32_t h =
        Hash(static_cast<uint32_t>(tx) * 374761393u ^
            static_cast<uint32_t>(ty) * 668265263u);

    float noise =
        static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

    // Subtle variation around neutral gray.
    float value = 0.50f + noise * 0.10f;
    value = std::clamp(value, 0.25f, 0.75f);

    BYTE c = static_cast<BYTE>(value * 255.0f);

    // Premultiplied alpha is required by UpdateLayeredWindow.
    const BYTE a = g_alpha;
    const BYTE pc = static_cast<BYTE>(
        (static_cast<unsigned int>(c) * a + 127) / 255);

    pixels[y * g_width + x] = { pc, pc, pc, a };
}
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static LRESULT CALLBACK WndProc(
    HWND h, UINT msg, WPARAM wParam, LPARAM lParam)
{
    switch (msg) {
    case WM_NCHITTEST:
        // The overlay never receives mouse clicks.
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();
        return 0;
    }
}

```

```

    case WM_HOTKEY:
        switch (wParam) {
            case 1: // F6
                g_enabled = !g_enabled;
                PresentLayer();
                break;

            case 2: // F7
                g_alpha = static_cast<BYTE>(
                    std::min(100, static_cast<int>(g_alpha) + 5));
                PresentLayer();
                break;

            case 3: // F8
                g_alpha = static_cast<BYTE>(
                    std::max(0, static_cast<int>(g_alpha) - 5));
                PresentLayer();
                break;
        }
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"DeckleWindowsOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"DeckleWindows",
        WS_POPUP,
        g_x, g_y, g_width, g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    RegisterHotKey(g_hwnd, 1, MOD_NOREPEAT, VK_F6);
    RegisterHotKey(g_hwnd, 2, MOD_NOREPEAT, VK_F7);
    RegisterHotKey(g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);

    DestroyLayerBitmap();
    return 0;
}

```

10.4. Deckle Windows v0.4

```
#include <windows.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")

static HWND g_hwnd = nullptr;
static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34; // ~13% opacity
static bool g_enabled = true;

struct Pixel {
    BYTE b, g, r, a;
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void DestroyLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    DestroyLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height; // top-down
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        DestroyLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
}
```

```

    return true;
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);

    // Generate a 1024x1024 procedural paper tile and repeat it over
    // the virtual desktop. The overlay itself is the only thing rendered;
    // the underlying screen is never captured.
    constexpr int TILE = 1024;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            if (!g_enabled || g_alpha == 0) {
                pixels[y * g_width + x] = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            // Subtle variation around neutral gray.
            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            BYTE c = static_cast<BYTE>(value * 255.0f);

            // Premultiplied alpha is required by UpdateLayeredWindow.
            const BYTE a = g_alpha;
            const BYTE pc = static_cast<BYTE>(
                (static_cast<unsigned int>(c) * a + 127) / 255);

            pixels[y * g_width + x] = { pc, pc, pc, a };
        }
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;
}

```

```

    PositionWindow();
    PresentLayer();
}

static LRESULT CALLBACK WndProc(
    HWND h, UINT msg, WPARAM wParam, LPARAM lParam)
{
    switch (msg) {
    case WM_NCHITTEST:
        // The overlay never receives mouse clicks.
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();
        return 0;

    case WM_HOTKEY:
        switch (wParam) {
        case 1: // F6
            g_enabled = !g_enabled;
            PresentLayer();
            break;

        case 2: // F7
            g_alpha = static_cast<BYTE>(
                std::min(100, static_cast<int>(g_alpha) + 5));
            PresentLayer();
            break;

        case 3: // F8
            g_alpha = static_cast<BYTE>(
                std::max(0, static_cast<int>(g_alpha) - 5));
            PresentLayer();
            break;

        case 4: // Esc
            PostQuitMessage(0);
            break;
        }
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEX wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"DeckleWindowsOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"DeckleWindows",
        WS_POPUP,
        g_x, g_y, g_width, g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    RegisterHotKey(g_hwnd, 1, MOD_NOREPEAT, VK_F6);
}

```

```

RegisterHotKey(g_hwnd, 2, MOD_NOREPEAT, VK_F7);
RegisterHotKey(g_hwnd, 3, MOD_NOREPEAT, VK_F8);
RegisterHotKey(g_hwnd, 4, MOD_NOREPEAT, VK_ESCAPE);

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

DestroyLayerBitmap();
return 0;
}

```

10.5. Deckle Windows v0.5

```

#include <windows.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;
constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_EXIT = 1002;
constexpr UINT IDM_OPACITY_UP = 1003;
constexpr UINT IDM_OPACITY_DOWN = 1004;
static NOTIFYICONDATAW g_nid = {};

static HWND g_hwnd = nullptr;
static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34; // ~13% opacity
static bool g_enabled = true;

struct Pixel {
    BYTE b, g, r, a;
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void DestroyLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    DestroyLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;

```

```

    bi.bmiHeader.biHeight = -g_height; // top-down
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        DestroyLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);

    return true;
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);

    // Generate a 1024x1024 procedural paper tile and repeat it over
    // the virtual desktop. The overlay itself is the only thing rendered;
    // the underlying screen is never captured.
    constexpr int TILE = 1024;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            if (!g_enabled || g_alpha == 0) {
                pixels[y * g_width + x] = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            // Subtle variation around neutral gray.
            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            BYTE c = static_cast<BYTE>(value * 255.0f);

            // Premultiplied alpha is required by UpdateLayeredWindow.
            const BYTE a = g_alpha;
            const BYTE pc = static_cast<BYTE>(
                (static_cast<unsigned int>(c) * a + 127) / 255);

            pixels[y * g_width + x] = { pc, pc, pc, a };
        }
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    BOOL ok = UpdateLayeredWindow(
        g_hwnd,

```

```

        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

ReleaseDC(nullptr, screenDC);
return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));
    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);
    lstrcpynW(g_nid.szTip, L"DeckleWindows", ARRAYSIZE(g_nid.szTip));
    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disattiva overlay" : L"Attiva overlay");

    AppendMenuW(menu, MF_STRING, IDM_OPACITY_UP, L"Aumenta intensità");
    AppendMenuW(menu, MF_STRING, IDM_OPACITY_DOWN, L"Diminuisci intensità");
    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);
    AppendMenuW(menu, MF_STRING, IDM_EXIT, L"Esci");

    POINT pt{};
    GetCursorPos(&pt);

    // Required so the menu closes correctly when the user clicks elsewhere.
    SetForegroundWindow(g_hwnd);
    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON | TPM_BOTTOMALIGN | TPM_LEFTALIGN,
        pt.x, pt.y, 0, g_hwnd, nullptr);

    DestroyMenu(menu);
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static LRESULT CALLBACK WndProc(
    HWND h, UINT msg, WPARAM wParam, LPARAM lParam)

```

```

{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN || lParam == WM_LBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;
        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;
        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;
        case IDM_EXIT:
            DestroyWindow(h);
            return 0;
        }
        break;

    case WM_NCHITTEST:
        // The overlay never receives mouse clicks.
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();
        return 0;

    case WM_HOTKEY:
        switch (wParam) {
        case 1: // F6
            g_enabled = !g_enabled;
            PresentLayer();
            break;

        case 2: // F7
            g_alpha = static_cast<BYTE>(
                std::min(100, static_cast<int>(g_alpha) + 5));
            PresentLayer();
            break;

        case 3: // F8
            ChangeOpacity(-5);
            break;

        case 4: // Ctrl+Alt+Shift+Q
            DestroyWindow(h);
            break;
        }
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEX wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"DeckleWindowsOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassEx(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,

```

```

        wc.lpszClassName,
        L"DeckleWindows",
        WS_POPUP,
        g_x, g_y, g_width, g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(g_hwnd, 1, MOD_NOREPEAT, VK_F6);
RegisterHotKey(g_hwnd, 2, MOD_NOREPEAT, VK_F7);
RegisterHotKey(g_hwnd, 3, MOD_NOREPEAT, VK_F8);
RegisterHotKey(g_hwnd, 4, MOD_CONTROL | MOD_ALT | MOD_SHIFT | MOD_NOREPEAT, 'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
DestroyLayerBitmap();
return 0;
}

```

10.6. Deckle Windows v0.6

```

#include <windows.h>
#include <shellapi.h>
#include <algorithm>
#include <stdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;
constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_EXIT = 1002;
constexpr UINT IDM_OPACITY_UP = 1003;
constexpr UINT IDM_OPACITY_DOWN = 1004;
static NOTIFYICONDATAW g_nid = {};

static HWND g_hwnd = nullptr;
static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34; // ~13% opacity
static bool g_enabled = true;

struct Pixel {
    BYTE b, g, r, a;
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void DestroyLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
    }
}

```

```

    g_bitmap = nullptr;
}

g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    DestroyLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height; // top-down
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        DestroyLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);

    return true;
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);

    // Generate a 1024x1024 procedural paper tile and repeat it over
    // the virtual desktop. The overlay itself is the only thing rendered;
    // the underlying screen is never captured.
    constexpr int TILE = 1024;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            if (!g_enabled || g_alpha == 0) {
                pixels[y * g_width + x] = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            // Subtle variation around neutral gray.
            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            BYTE c = static_cast<BYTE>(value * 255.0f);

            // Premultiplied alpha is required by UpdateLayeredWindow.
            const BYTE a = g_alpha;
            const BYTE pc = static_cast<BYTE>(
                (static_cast<unsigned int>(c) * a + 127) / 255);

            pixels[y * g_width + x] = { pc, pc, pc, a };
        }
    }
}

```

```

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));
    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);
    wcscpy_s(g_nid.szTip, L"DeckleWindows");
    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disattiva overlay" : L"Attiva overlay");

    AppendMenuW(menu, MF_STRING, IDM_OPACITY_UP, L"Aumenta intensità");
    AppendMenuW(menu, MF_STRING, IDM_OPACITY_DOWN, L"Diminuisci intensità");
    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);
    AppendMenuW(menu, MF_STRING, IDM_EXIT, L"Esci");

    POINT pt{};
    GetCursorPos(&pt);

    // Required so the menu closes correctly when the user clicks elsewhere.
    SetForegroundWindow(g_hwnd);
    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON | TPM_BOTTOMALIGN | TPM_LEFTALIGN,

```

```

        pt.x, pt.y, 0, g_hwnd, nullptr);
    DestroyMenu(menu);
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static LRESULT CALLBACK WndProc(
    HWND h, UINT msg, WPARAM wParam, LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN || lParam == WM_LBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDBLCLK)
            ToggleOverlay();
        return 0;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;
        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;
        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;
        case IDM_EXIT:
            DestroyWindow(h);
            return 0;
        }
        break;

    case WM_NCHITTEST:
        // The overlay never receives mouse clicks.
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();
        return 0;

    case WM_HOTKEY:
        switch (wParam) {
        case 1: // F6
            g_enabled = !g_enabled;
            PresentLayer();
            break;

        case 2: // F7
            g_alpha = static_cast<BYTE>(
                std::min(100, static_cast<int>(g_alpha) + 5));
            PresentLayer();
            break;

        case 3: // F8
            ChangeOpacity(-5);
            break;

        case 4: // Ctrl+Alt+Shift+Q
            DestroyWindow(h);
            break;
        }
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEX wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;

```

```

wc.lpszClassName = L"DeckleWindowsOverlay";
wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

if (!RegisterClassExW(&wc))
    return 1;

g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

const DWORD exStyle =
    WS_EX_LAYERED |
    WS_EX_TRANSPARENT |
    WS_EX_NOACTIVATE |
    WS_EX_TOOLWINDOW |
    WS_EX_TOPMOST;

g_hwnd = CreateWindowExW(
    exStyle,
    wc.lpszClassName,
    L"DeckleWindows",
    WS_POPUP,
    g_x, g_y, g_width, g_height,
    nullptr,
    nullptr,
    instance,
    nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(g_hwnd, 1, MOD_NOREPEAT, VK_F6);
RegisterHotKey(g_hwnd, 2, MOD_NOREPEAT, VK_F7);
RegisterHotKey(g_hwnd, 3, MOD_NOREPEAT, VK_F8);
RegisterHotKey(g_hwnd, 4, MOD_CONTROL | MOD_ALT | MOD_SHIFT | MOD_NOREPEAT, 'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
DestroyLayerBitmap();
return 0;
}

```

10.7. GrainLayer v0.8

```

#include <windows.h>
#include <shellapi.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

```

```

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {

```

```

        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);
            p.g = static_cast<BYTE>(
                (static_cast<unsigned int>(g) * a + 127) / 255);
            p.b = static_cast<BYTE>(
                (static_cast<unsigned int>(b) * a + 127) / 255);
            p.a = a;
        }
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;

```

```

blend.SourceConstantAlpha = 255;
blend.AlphaFormat = AC_SRC_ALPHA;

const BOOL ok = UpdateLayeredWindow(
    g_hwnd,
    screenDC,
    &dst,
    &size,
    g_memDC,
    &src,
    0,
    &blend,
    ULW_ALPHA);

ReleaseDC(nullptr, screenDC);
return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);
}

```

```

        InsertMenuItemW(
            menu,
            GetMenuItemCount(menu),
            TRUE,
            &mii);
    }

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");
    AppendTintItem(menu, kTints[1]);
    AppendTintItem(menu, kTints[2]);
    AppendTintItem(menu, kTints[3]);
    AppendTintItem(menu, kTints[4]);
    AppendTintItem(menu, kTints[5]);
    AppendTintItem(menu, kTints[6]);
    AppendTintItem(menu, kTints[7]);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

    DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

```

```

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(const MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT*>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT*>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU &&
                FindTint(mis->itemID)) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);
            }
        }
    }
}

```

```

        MeasureTintMenuItem(mutableMis);
        return TRUE;
    }
}
break;

case WM_DRAWITEM:
    if (wParam == 0) {
        const auto* dis =
            reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

        if (dis && dis->CtlType == ODT_MENU &&
            FindTint(dis->itemID)) {
            DrawTintMenuItem(dis);
            return TRUE;
        }
    }
    break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)

```

```

{
    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);
    UnregisterHotKey(g_hwnd, 4);

    RemoveTrayIcon();
    SafeDeleteLayerBitmap();

    return 0;
}

```

10.8. GrainLayer v0.9

```

#include <windows.h>
#include <shellapi.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

```

```

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;

```

```

    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);
            p.g = static_cast<BYTE>(
                (static_cast<unsigned int>(g) * a + 127) / 255);
            p.b = static_cast<BYTE>(

```

```

        (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{

```

```

    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");
    AppendTintItem(menu, kTints[1]);
    AppendTintItem(menu, kTints[2]);
    AppendTintItem(menu, kTints[3]);
    AppendTintItem(menu, kTints[4]);
    AppendTintItem(menu, kTints[5]);
    AppendTintItem(menu, kTints[6]);
    AppendTintItem(menu, kTints[7]);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

    DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

```

```

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
            ? GetSysColor(COLOR_HIGHLIGHT)
            : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
            ? GetSysColor(COLOR_HIGHLIGHTTEXT)
            : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(

```

```

HWND h,
UINT msg,
WPARAM wParam,
LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONUP)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU &&
                FindTint(mis->itemID)) {
                MeasureTintMenuItem(
                    const_cast<MEASUREITEMSTRUCT*>(mis));
                return TRUE;
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU &&
                FindTint(dis->itemID)) {
                DrawTintMenuItem(dis);
                return TRUE;
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

    case WM_NCHITTEST:
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();
        return 0;

    case WM_HOTKEY:
        switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);

```

```

        break;

    case 4:
        DestroyWindow(h);
        break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);
    UnregisterHotKey(g_hwnd, 4);
}

```

```

    RemoveTrayIcon();
    SafeDeleteLayerBitmap();

    return 0;
}

```

10.9. GrainLayer v1.0

```

#include <windows.h>
#include <shellapi.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
}

```

```

    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

```

```

        if (!g_enabled || g_alpha == 0) {
            p = { 0, 0, 0, 0 };
            continue;
        }

        const int tx = x & (TILE - 1);
        const int ty = y & (TILE - 1);

        const uint32_t h =
            Hash(static_cast<uint32_t>(tx) * 374761393u ^
                static_cast<uint32_t>(ty) * 668265263u);

        const float noise =
            static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

        float value = 0.50f + noise * 0.10f;
        value = std::clamp(value, 0.25f, 0.75f);

        const BYTE gray =
            static_cast<BYTE>(value * 255.0f);

        BYTE r = gray;
        BYTE g = gray;
        BYTE b = gray;

        if (g_hasTint) {
            r = BlendChannel(gray, tintR, TINT_MIX);
            g = BlendChannel(gray, tintG, TINT_MIX);
            b = BlendChannel(gray, tintB, TINT_MIX);
        }

        // Premultiplied alpha required by UpdateLayeredWindow.
        const BYTE a = g_alpha;

        p.r = static_cast<BYTE>(
            (static_cast<unsigned int>(r) * a + 127) / 255);
        p.g = static_cast<BYTE>(
            (static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>(
            (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

```

```

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]); // Foxed Amber
    }
}

```

```

        AppendTintItem(warmMenu, kTints[2]); // Bookcloth
        AppendTintItem(warmMenu, kTints[3]); // Recycled Kraft
        AppendTintItem(warmMenu, kTints[4]); // Plum Kozo
        AppendTintItem(warmMenu, kTints[5]); // Rose Quartz
        AppendTintItem(warmMenu, kTints[6]); // Sage Press
        AppendTintItem(warmMenu, kTints[7]); // Nordic Sky
        AppendMenuW(menu, MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]); // Soft Wave
        AppendTintItem(paperMenu, kTints[9]); // Rice Paper
        AppendTintItem(paperMenu, kTints[10]); // Laid Cotton
        AppendTintItem(paperMenu, kTints[11]); // Newsprint
        AppendTintItem(paperMenu, kTints[12]); // Cold Press
        AppendTintItem(paperMenu, kTints[13]); // Artist Canvas
        AppendTintItem(paperMenu, kTints[14]); // Felt Side
        AppendTintItem(paperMenu, kTints[15]); // Frost Glassine
        AppendMenuW(menu, MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

    DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected

```

```

        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

HBRUSH bg = CreateSolidBrush(background);
FillRect(dc, &rc, bg);
DeleteObject(bg);

DrawTintSwatch(dc, rc, tint->color);

SetBkMode(dc, TRANSPARENT);
SetTextColor(dc, text);

RECT textRc = rc;
textRc.left += 34;
textRc.right -= 8;

DrawTextW(
    dc,
    tint->name,
    -1,
    &textRc,
    DT_SINGLELINE |
    DT_VCENTER |
    DT_LEFT |
    DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU &&
                FindTint(mis->itemID)) {
                MeasureTintMenuItem(
                    const_cast<MEASUREITEMSTRUCT*>(mis));
                return TRUE;
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU &&
                FindTint(dis->itemID)) {
                DrawTintMenuItem(dis);
                return TRUE;
            }
        }
    }
}

```

```

        break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEX wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =

```

```

WS_EX_LAYERED |
WS_EX_TRANSPARENT |
WS_EX_NOACTIVATE |
WS_EX_TOOLWINDOW |
WS_EX_TOPMOST;

g_hwnd = CreateWindowExW(
    exStyle,
    wc.lpszClassName,
    L"GrainLayer",
    WS_POPUP,
    g_x,
    g_y,
    g_width,
    g_height,
    nullptr,
    nullptr,
    instance,
    nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(
    g_hwnd, 1, MOD_NOREPEAT, VK_F6);

RegisterHotKey(
    g_hwnd, 2, MOD_NOREPEAT, VK_F7);

RegisterHotKey(
    g_hwnd, 3, MOD_NOREPEAT, VK_F8);

RegisterHotKey(
    g_hwnd,
    4,
    MOD_CONTROL |
    MOD_ALT |
    MOD_SHIFT |
    MOD_NOREPEAT,
    'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.10. GrainLayer v1.0.1

```

#include <windows.h>
#include <shellapi.h>
#include <algorithm>
#include <stdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;

```

```

constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;

```

```

    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);

```

```

        p.g = static_cast<BYTE>((static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>((static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>((std::clamp(static_cast<int>(g_alpha) + delta, 0, 100)));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcsncpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

```

```

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]); // Foxed Amber
        AppendTintItem(warmMenu, kTints[2]); // Bookcloth
        AppendTintItem(warmMenu, kTints[3]); // Recycled Kraft
        AppendTintItem(warmMenu, kTints[4]); // Plum Kozo
        AppendTintItem(warmMenu, kTints[5]); // Rose Quartz
        AppendTintItem(warmMenu, kTints[6]); // Sage Press
        AppendTintItem(warmMenu, kTints[7]); // Nordic Sky
        AppendMenuW(menu, MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]); // Soft Wave
        AppendTintItem(paperMenu, kTints[9]); // Rice Paper
        AppendTintItem(paperMenu, kTints[10]); // Laid Cotton
        AppendTintItem(paperMenu, kTints[11]); // Newsprint
        AppendTintItem(paperMenu, kTints[12]); // Cold Press
        AppendTintItem(paperMenu, kTints[13]); // Artist Canvas
        AppendTintItem(paperMenu, kTints[14]); // Felt Side
        AppendTintItem(paperMenu, kTints[15]); // Frost Glassine
        AppendMenuW(menu, MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);
}

```

```

SetForegroundWindow(g_hwnd);

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;
}

```

```

HDC dc = GetDC(g_hwnd);
HFONT font = static_cast<HFONT>(
    GetStockObject(DEFAULT_GUI_FONT));

HFONT old = static_cast<HFONT>(
    SelectObject(dc, font));

SIZE size = {};
GetTextExtentPoint32W(
    dc,
    tint->name,
    lstrlenW(tint->name),
    &size);

SelectObject(dc, old);
ReleaseDC(g_hwnd, dc);

mis->itemWidth =
    static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU &&
                FindTint(mis->itemID)) {
                MeasureTintMenuItem(
                    const_cast<MEASUREITEMSTRUCT*>(mis));
                return TRUE;
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU &&
                FindTint(dis->itemID)) {
                DrawTintMenuItem(dis);
                return TRUE;
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
            {
                const TintMenuItem* tint =
                    FindTint(LOWORD(wParam));

                if (tint) {
                    SetTint(tint->color, true);
                    return 0;
                }
            }
        }
        break;
    }
    break;
}

```

```

        case WM_NCHITTEST:
            return HTTRANSPARENT;

        case WM_MOUSEACTIVATE:
            return MA_NOACTIVATE;

        case WM_DISPLAYCHANGE:
            Refresh();
            return 0;

        case WM_HOTKEY:
            switch (wParam) {
                case 1:
                    ToggleOverlay();
                    break;

                case 2:
                    ChangeOpacity(+5);
                    break;

                case 3:
                    ChangeOpacity(-5);
                    break;

                case 4:
                    DestroyWindow(h);
                    break;
            }
            return 0;

        case WM_DESTROY:
            PostQuitMessage(0);
            return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    WNDCLASSEX wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(

```

```

        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.11. GrainLayer v1.1

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;

```

```

};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }
}

```

```

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);
            p.g = static_cast<BYTE>(
                (static_cast<unsigned int>(g) * a + 127) / 255);
            p.b = static_cast<BYTE>(
                (static_cast<unsigned int>(b) * a + 127) / 255);
            p.a = a;
        }
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(

```

```

        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WM_APP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),

```

```

        TRUE,
        &mii);
}

static void UpdateIntensityLabel()
{
    if (!g_intensityLabel)
        return;

    wchar_t text[64] = {};
    swprintf_s(
        text,
        L"Intensity: %d%%",
        static_cast<int>(
            (static_cast<int>(g_alpha) * 100 + 127) / 255));

    SetWindowTextW(g_intensityLabel, text);
}

static void CloseIntensityPopup()
{
    if (g_intensityPopup) {
        DestroyWindow(g_intensityPopup);
        g_intensityPopup = nullptr;
        g_intensitySlider = nullptr;
        g_intensityLabel = nullptr;
    }
}

static LRESULT CALLBACK IntensityPopupProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_CREATE:
    {
        g_intensityLabel = CreateWindowExW(
            0,
            L"STATIC",
            L"Intensity: 13%",
            WS_CHILD | WS_VISIBLE,
            18, 10, 220, 20,
            h,
            nullptr,
            GetModuleHandleW(nullptr),
            nullptr);

        g_intensitySlider = CreateWindowExW(
            0,
            TRACKBAR_CLASSW,
            L"",
            WS_CHILD | WS_VISIBLE |
            TBS_AUTOTICKS | TBS_HORZ,
            14, 34, 232, 34,
            h,
            nullptr,
            GetModuleHandleW(nullptr),
            nullptr);

        SendMessageW(
            g_intensitySlider,
            TBM_SETRANGE,
            TRUE,
            MAKELONG(0, 100));

        const int percent =
            (static_cast<int>(g_alpha) * 100 + 127) / 255;

        SendMessageW(
            g_intensitySlider,
            TBM_SETPOS,
            TRUE,
            percent);

        UpdateIntensityLabel();
        return 0;
    }

    case WM_HSCROLL:
        if (reinterpret_cast<HWND>(lParam) == g_intensitySlider) {
            const int percent = static_cast<int>(
                SendMessageW(
                    g_intensitySlider,
                    TBM_GETPOS,
                    0,
                    0));

            // 0..100% is the user-facing intensity range.
            // Map it to the same alpha range used by the existing renderer.
            g_alpha = static_cast<BYTE>(
                std::clamp(
                    static_cast<int>(
                        (percent * 255 + 50) / 100),
                    0,
                    255));
        }
    }
}

```

```

        UpdateIntensityLabel();
        PresentLayer();
    }
    return 0;

case WM_ACTIVATE:
    if (LOWORD(wParam) == WA_INACTIVE) {
        CloseIntensityPopup();
        return 0;
    }
    break;

case WM_NCHITTEST:
    return HTCLIENT;

case WM_CLOSE:
    CloseIntensityPopup();
    return 0;
}

return DefWindowProcW(h, msg, wParam, lParam);
}

static void ShowIntensityPopup()
{
    CloseIntensityPopup();

    HINSTANCE instance = GetModuleHandleW(nullptr);

    static bool registered = false;

    if (!registered) {
        WNDCLASSEX wc = {};
        wc.cbSize = sizeof(wc);
        wc.hInstance = instance;
        wc.lpfnWndProc = IntensityPopupProc;
        wc.lpszClassName = L"GrainLayerIntensityPopup";
        wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
        wc.hbrBackground =
            reinterpret_cast<HBRUSH>(
                static_cast<INT_PTR>(COLOR_MENU + 1));

        if (!RegisterClassExW(&wc) &&
            GetLastError() != ERROR_CLASS_ALREADY_EXISTS) {
            return;
        }

        registered = true;
    }

    POINT pt = {};
    GetCursorPos(&pt);

    // Position the popup above the cursor/tray area when possible.
    int x = pt.x - INTENSITY_POPUP_W + 16;
    int y = pt.y - INTENSITY_POPUP_H - 8;

    const int screenW = GetSystemMetrics(SM_CXSCREEN);
    const int screenH = GetSystemMetrics(SM_CYSCREEN);

    x = std::clamp(x, 0, std::max(0, screenW - INTENSITY_POPUP_W));
    y = std::clamp(y, 0, std::max(0, screenH - INTENSITY_POPUP_H));

    g_intensityPopup = CreateWindowExW(
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST |
        WS_EX_NOACTIVATE,
        L"GrainLayerIntensityPopup",
        L"Intensity",
        WS_POPUP | WS_BORDER,
        x, y,
        INTENSITY_POPUP_W,
        INTENSITY_POPUP_H,
        g_hwnd,
        nullptr,
        instance,
        nullptr);

    if (!g_intensityPopup)
        return;

    ShowWindow(g_intensityPopup, SW_SHOWNOACTIVATE);
    UpdateWindow(g_intensityPopup);
    SetForegroundWindow(g_intensityPopup);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");
}

```

```

AppendMenuW(
    menu,
    MF_STRING,
    IDM_OPACITY_UP,
    L"Increase intensity (F7)");

AppendMenuW(
    menu,
    MF_STRING,
    IDM_OPACITY_DOWN,
    L"Decrease intensity (F8)");

AppendMenuW(
    menu,
    MF_STRING,
    IDM_INTENSITY,
    L"Intensity...");

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

HMENU warmMenu = CreatePopupMenu();
if (warmMenu) {
    AppendTintItem(warmMenu, kTints[1]); // Foxed Amber
    AppendTintItem(warmMenu, kTints[2]); // Bookcloth
    AppendTintItem(warmMenu, kTints[3]); // Recycled Kraft
    AppendTintItem(warmMenu, kTints[4]); // Plum Kozo
    AppendTintItem(warmMenu, kTints[5]); // Rose Quartz
    AppendTintItem(warmMenu, kTints[6]); // Sage Press
    AppendTintItem(warmMenu, kTints[7]); // Nordic Sky
    AppendMenuW(menu, MF_POPUP,
        reinterpret_cast<UINT_PTR>(warmMenu),
        L"Warm & tinted");
}

HMENU paperMenu = CreatePopupMenu();
if (paperMenu) {
    AppendTintItem(paperMenu, kTints[8]); // Soft Wave
    AppendTintItem(paperMenu, kTints[9]); // Rice Paper
    AppendTintItem(paperMenu, kTints[10]); // Laid Cotton
    AppendTintItem(paperMenu, kTints[11]); // Newsprint
    AppendTintItem(paperMenu, kTints[12]); // Cold Press
    AppendTintItem(paperMenu, kTints[13]); // Artist Canvas
    AppendTintItem(paperMenu, kTints[14]); // Felt Side
    AppendTintItem(paperMenu, kTints[15]); // Frost Glassine
    AppendMenuW(menu, MF_POPUP,
        reinterpret_cast<UINT_PTR>(paperMenu),
        L"Paper");
}

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(
    menu,
    MF_STRING,
    IDM_EXIT,
    L"Exit (Ctrl+Alt+Shift+Q)");

POINT pt = {};
GetCursorPos(&pt);

SetForegroundWindow(g_hwnd);

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(

```

```

        GetStockObject(GRAY_BRUSH));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNBLCLK)
            ToggleOverlay();
        return 0;
    }
}

```

```

case WM_MEASUREITEM:
    if (wParam == 0) {
        const auto* mis =
            reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

        if (mis && mis->CtlType == ODT_MENU &&
            FindTint(mis->itemID)) {
            MeasureTintMenuItem(
                const_cast<MEASUREITEMSTRUCT*>(mis));
            return TRUE;
        }
    }
    break;

case WM_DRAWITEM:
    if (wParam == 0) {
        const auto* dis =
            reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

        if (dis && dis->CtlType == ODT_MENU &&
            FindTint(dis->itemID)) {
            DrawTintMenuItem(dis);
            return TRUE;
        }
    }
    break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
    case IDM_TOGGLE:
        ToggleOverlay();
        return 0;

    case IDM_OPACITY_UP:
        ChangeOpacity(+5);
        return 0;

    case IDM_OPACITY_DOWN:
        ChangeOpacity(-5);
        return 0;

    case IDM_INTENSITY:
        ShowIntensityPopup();
        return 0;

    case IDM_EXIT:
        DestroyWindow(h);
        return 0;

    case IDM_TINT_NONE:
        SetTint(RGB(255, 255, 255), false);
        return 0;

    default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
    case 1:
        ToggleOverlay();
        break;

    case 2:
        ChangeOpacity(+5);
        break;

    case 3:
        ChangeOpacity(-5);
        break;

    case 4:
        DestroyWindow(h);
        break;
    }
    return 0;

case WM_DESTROY:

```

```

        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);
    UnregisterHotKey(g_hwnd, 4);

    RemoveTrayIcon();
    SafeDeleteLayerBitmap();
}

```

```

    return 0;
}

```

10.12. GrainLayer v1.2

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdlib>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
};

```

```

    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);

```

```

const BYTE tintB = GetBValue(g_tint);

// This controls how much the selected pastel affects the paper grain.
// Kept deliberately low so the tint remains subtle.
constexpr float TINT_MIX = 0.22f;

for (int y = 0; y < g_height; ++y) {
    for (int x = 0; x < g_width; ++x) {
        Pixel& p = pixels[y * g_width + x];

        if (!g_enabled || g_alpha == 0) {
            p = { 0, 0, 0, 0 };
            continue;
        }

        const int tx = x & (TILE - 1);
        const int ty = y & (TILE - 1);

        const uint32_t h =
            Hash(static_cast<uint32_t>(tx) * 374761393u ^
                static_cast<uint32_t>(ty) * 668265263u);

        const float noise =
            static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

        float value = 0.50f + noise * 0.10f;
        value = std::clamp(value, 0.25f, 0.75f);

        const BYTE gray =
            static_cast<BYTE>(value * 255.0f);

        BYTE r = gray;
        BYTE g = gray;
        BYTE b = gray;

        if (g_hasTint) {
            r = BlendChannel(gray, tintR, TINT_MIX);
            g = BlendChannel(gray, tintG, TINT_MIX);
            b = BlendChannel(gray, tintB, TINT_MIX);
        }

        // Premultiplied alpha required by UpdateLayeredWindow.
        const BYTE a = g_alpha;

        p.r = static_cast<BYTE>(
            (static_cast<unsigned int>(r) * a + 127) / 255);
        p.g = static_cast<BYTE>(
            (static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>(
            (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

```

```

}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcsncpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))

```

```

        return;

POINT pt = {};
GetCursorPos(&pt);

const int left = itemRect.left + 28;
const int right = itemRect.right - 18;

if (right <= left)
    return;

const int clampedX = std::clamp(pt.x, left, right);
const int percent =
    ((clampedX - left) * 100 + (right - left) / 2) /
    (right - left);

const BYTE oldAlpha = g_alpha;
PercentToAlpha(percent);

if (oldAlpha != g_alpha)
    PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity                %d%%",
        AlphaToPercent());

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

```

```

HBRUSH fillBrush = CreateSolidBrush(
    GetSysColor(COLOR_HIGHLIGHT));
FillRect(dc, &filled, fillBrush);
DeleteObject(fillBrush);

const int radius = 7;
RECT knob = {
    knobX - radius,
    trackY - radius,
    knobX + radius + 1,
    trackY + radius + 1
};

HBRUSH knobBrush = CreateSolidBrush(face);
FillRect(dc, &knob, knobBrush);
DeleteObject(knobBrush);

FrameRect(
    dc,
    &knob,
    static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static VOID CALLBACK MenuSliderTimer(
    HWND,
    UINT,
    UINT_PTR,
    DWORD)
{
    if (!g_menuTracking)
        return;

    // The timer fires only while TrackPopupMenu's modal loop is active.
    // Find the currently active popup menu window.
    HWND menuWindow = FindWindowW(L"#32768", nullptr);
    if (!menuWindow)
        return;

    // Get the menu associated with the visible popup is not directly exposed
    // by Win32. The active menu item rectangle is therefore located using
    // screen geometry stored by the tray menu routine below.
}

static HMENU g_activeMenu = nullptr;

static VOID CALLBACK ActiveMenuTimer(
    HWND,
    UINT,
    UINT_PTR,
    DWORD)
{
    if (g_menuTracking && g_activeMenu)
        SetIntensityFromCursor(g_activeMenu);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
    }
}

```

```

        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    g_activeMenu = menu;
    g_menuTracking = true;

    const UINT_PTR timerId =
        SetTimer(g_hwnd, 99, 15, ActiveMenuTimer);

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

    if (timerId)
        KillTimer(g_hwnd, timerId);

    g_menuTracking = false;
    g_activeMenu = nullptr;

    DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;
}

```

```

HDC dc = dis->hDC;
RECT rc = dis->rcItem;

const bool selected =
    (dis->itemState & ODS_SELECTED) != 0;

const COLORREF background =
    selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

const COLORREF text =
    selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

HBRUSH bg = CreateSolidBrush(background);
FillRect(dc, &rc, bg);
DeleteObject(bg);

DrawTintSwatch(dc, rc, tint->color);

SetBkMode(dc, TRANSPARENT);
SetTextColor(dc, text);

RECT textRc = rc;
textRc.left += 34;
textRc.right -= 8;

DrawTextW(
    dc,
    tint->name,
    -1,
    &textRc,
    DT_SINGLELINE |
    DT_VCENTER |
    DT_LEFT |
    DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT*>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT*>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }
            }
        }
    }
}

```

```

        }

        if (FindTint(mis->itemID)) {
            MeasureTintMenuItem(mutableMis);
            return TRUE;
        }
    }
}
break;

case WM_DRAWITEM:
    if (wParam == 0) {
        const auto* dis =
            reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

        if (dis && dis->CtlType == ODT_MENU) {
            if (dis->itemID == IDM_INTENSITY_SLIDER) {
                DrawIntensitySlider(dis);
                return TRUE;
            }

            if (FindTint(dis->itemID)) {
                DrawTintMenuItem(dis);
                return TRUE;
            }
        }
    }
}
break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
    }
    break;
}
break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;

```

```

    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);
    UnregisterHotKey(g_hwnd, 4);

    RemoveTrayIcon();
    SafeDeleteLayerBitmap();

    return 0;
}

```

10.13. GrainLayer v1.2.1

```
#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <stdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
};
```

```

    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

```

```

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

```

```

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

```

```

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

```

```

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

```

```

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.

```

```

// Kept deliberately low so the tint remains subtle.
constexpr float TINT_MIX = 0.22f;

for (int y = 0; y < g_height; ++y) {
    for (int x = 0; x < g_width; ++x) {
        Pixel& p = pixels[y * g_width + x];

        if (!g_enabled || g_alpha == 0) {
            p = { 0, 0, 0, 0 };
            continue;
        }

        const int tx = x & (TILE - 1);
        const int ty = y & (TILE - 1);

        const uint32_t h =
            Hash(static_cast<uint32_t>(tx) * 374761393u ^
                static_cast<uint32_t>(ty) * 668265263u);

        const float noise =
            static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

        float value = 0.50f + noise * 0.10f;
        value = std::clamp(value, 0.25f, 0.75f);

        const BYTE gray =
            static_cast<BYTE>(value * 255.0f);

        BYTE r = gray;
        BYTE g = gray;
        BYTE b = gray;

        if (g_hasTint) {
            r = BlendChannel(gray, tintR, TINT_MIX);
            g = BlendChannel(gray, tintG, TINT_MIX);
            b = BlendChannel(gray, tintB, TINT_MIX);
        }

        // Premultiplied alpha required by UpdateLayeredWindow.
        const BYTE a = g_alpha;

        p.r = static_cast<BYTE>(
            (static_cast<unsigned int>(r) * a + 127) / 255);
        p.g = static_cast<BYTE>(
            (static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>(
            (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()

```

```

{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};

```

```

GetCursorPos(&pt);

const int left = itemRect.left + 28;
const int right = itemRect.right - 18;

if (right <= left)
    return;

const int mouseX = static_cast<int>(pt.x);
const int clampedX =
    mouseX < left ? left :
    (mouseX > right ? right : mouseX);

const int percent =
    ((clampedX - left) * 100 + (right - left) / 2) /
    (right - left);

const BYTE oldAlpha = g_alpha;
PercentToAlpha(percent);

if (oldAlpha != g_alpha)
    PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity                %d%%",
        AlphaToPercent());

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

```

```

HBRUSH fillBrush = CreateSolidBrush(
    GetSysColor(COLOR_HIGHLIGHT));
FillRect(dc, &filled, fillBrush);
DeleteObject(fillBrush);

const int radius = 7;
RECT knob = {
    knobX - radius,
    trackY - radius,
    knobX + radius + 1,
    trackY + radius + 1
};

HBRUSH knobBrush = CreateSolidBrush(face);
FillRect(dc, &knob, knobBrush);
DeleteObject(knobBrush);

FrameRect(
    dc,
    &knob,
    static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static VOID CALLBACK MenuSliderTimer(
    HWND,
    UINT,
    UINT_PTR,
    DWORD)
{
    if (!g_menuTracking)
        return;

    // The timer fires only while TrackPopupMenu's modal loop is active.
    // Find the currently active popup menu window.
    HWND menuWindow = FindWindowW(L"#32768", nullptr);
    if (!menuWindow)
        return;

    // Get the menu associated with the visible popup is not directly exposed
    // by Win32. The active menu item rectangle is therefore located using
    // screen geometry stored by the tray menu routine below.
}

static HMENU g_activeMenu = nullptr;

static VOID CALLBACK ActiveMenuTimer(
    HWND,
    UINT,
    UINT_PTR,
    DWORD)
{
    if (g_menuTracking && g_activeMenu)
        SetIntensityFromCursor(g_activeMenu);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
    }
}

```

```

        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    g_activeMenu = menu;
    g_menuTracking = true;

    const UINT_PTR timerId =
        SetTimer(g_hwnd, 99, 15, ActiveMenuTimer);

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

    if (timerId)
        KillTimer(g_hwnd, timerId);

    g_menuTracking = false;
    g_activeMenu = nullptr;

    DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;
}

```

```

HDC dc = dis->hDC;
RECT rc = dis->rcItem;

const bool selected =
    (dis->itemState & ODS_SELECTED) != 0;

const COLORREF background =
    selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

const COLORREF text =
    selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

HBRUSH bg = CreateSolidBrush(background);
FillRect(dc, &rc, bg);
DeleteObject(bg);

DrawTintSwatch(dc, rc, tint->color);

SetBkMode(dc, TRANSPARENT);
SetTextColor(dc, text);

RECT textRc = rc;
textRc.left += 34;
textRc.right -= 8;

DrawTextW(
    dc,
    tint->name,
    -1,
    &textRc,
    DT_SINGLELINE |
    DT_VCENTER |
    DT_LEFT |
    DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT*>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT*>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }
            }
        }
    }
}

```

```

        }

        if (FindTint(mis->itemID)) {
            MeasureTintMenuItem(mutableMis);
            return TRUE;
        }
    }
}
break;

case WM_DRAWITEM:
    if (wParam == 0) {
        const auto* dis =
            reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

        if (dis && dis->CtlType == ODT_MENU) {
            if (dis->itemID == IDM_INTENSITY_SLIDER) {
                DrawIntensitySlider(dis);
                return TRUE;
            }

            if (FindTint(dis->itemID)) {
                DrawTintMenuItem(dis);
                return TRUE;
            }
        }
    }
}
break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
    }
    break;
}
break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;

```

```

    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);
    UnregisterHotKey(g_hwnd, 4);

    RemoveTrayIcon();
    SafeDeleteLayerBitmap();

    return 0;
}

```

10.14. GrainLayer v1.2.2

```
#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <stdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
};
```

```

    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

```

```

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

```

```

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

```

```

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

```

```

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

```

```

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.

```

```

// Kept deliberately low so the tint remains subtle.
constexpr float TINT_MIX = 0.22f;

for (int y = 0; y < g_height; ++y) {
    for (int x = 0; x < g_width; ++x) {
        Pixel& p = pixels[y * g_width + x];

        if (!g_enabled || g_alpha == 0) {
            p = { 0, 0, 0, 0 };
            continue;
        }

        const int tx = x & (TILE - 1);
        const int ty = y & (TILE - 1);

        const uint32_t h =
            Hash(static_cast<uint32_t>(tx) * 374761393u ^
                static_cast<uint32_t>(ty) * 668265263u);

        const float noise =
            static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

        float value = 0.50f + noise * 0.10f;
        value = std::clamp(value, 0.25f, 0.75f);

        const BYTE gray =
            static_cast<BYTE>(value * 255.0f);

        BYTE r = gray;
        BYTE g = gray;
        BYTE b = gray;

        if (g_hasTint) {
            r = BlendChannel(gray, tintR, TINT_MIX);
            g = BlendChannel(gray, tintG, TINT_MIX);
            b = BlendChannel(gray, tintB, TINT_MIX);
        }

        // Premultiplied alpha required by UpdateLayeredWindow.
        const BYTE a = g_alpha;

        p.r = static_cast<BYTE>(
            (static_cast<unsigned int>(r) * a + 127) / 255);
        p.g = static_cast<BYTE>(
            (static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>(
            (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()

```

```

{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};

```

```

GetCursorPos(&pt);

const int left = itemRect.left + 28;
const int right = itemRect.right - 18;

if (right <= left)
    return;

const int mouseX = static_cast<int>(pt.x);
const int clampedX =
    mouseX < left ? left :
    (mouseX > right ? right : mouseX);

const int percent =
    ((clampedX - left) * 100 + (right - left) / 2) /
    (right - left);

const BYTE oldAlpha = g_alpha;
PercentToAlpha(percent);

if (oldAlpha != g_alpha)
    PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity                %d%%",
        AlphaToPercent());

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

```

```

HBRUSH fillBrush = CreateSolidBrush(
    GetSysColor(COLOR_HIGHLIGHT));
FillRect(dc, &filled, fillBrush);
DeleteObject(fillBrush);

const int radius = 7;
RECT knob = {
    knobX - radius,
    trackY - radius,
    knobX + radius + 1,
    trackY + radius + 1
};

HBRUSH knobBrush = CreateSolidBrush(face);
FillRect(dc, &knob, knobBrush);
DeleteObject(knobBrush);

FrameRect(
    dc,
    &knob,
    static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
                case WM_LBUTTONDOWN:
                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        g_sliderDragging = true;
                        SetIntensityFromPoint(g_activeMenu, msg->pt);

                        // Consume the click so the menu stays open.
                        return 1;
                    }
                    break;
            }
        }
    }
}

```

```

        case WM_MOUSEMOVE:
            if (g_sliderDragging) {
                SetIntensityFromPoint(g_activeMenu, msg->pt);
                return 1;
            }

            if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                SetIntensityFromPoint(g_activeMenu, msg->pt);
            }
            break;

        case WM_LBUTTONDOWN:
            if (g_sliderDragging) {
                SetIntensityFromPoint(g_activeMenu, msg->pt);
                g_sliderDragging = false;

                // Consume the release so the menu stays open.
                return 1;
            }
            break;
    }
}

return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    {
        HMENU menu = CreatePopupMenu();
        if (!menu)
            return;

        AppendMenuW(
            menu,
            MF_STRING,
            IDM_TOGGLE,
            g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

        AppendMenuW(
            menu,
            MF_STRING,
            IDM_OPACITY_UP,
            L"Increase intensity (F7)");

        AppendMenuW(
            menu,
            MF_STRING,
            IDM_OPACITY_DOWN,
            L"Decrease intensity (F8)");

        AppendIntensitySlider(menu);

        AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

        AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

        HMENU warmMenu = CreatePopupMenu();
        if (warmMenu) {
            AppendTintItem(warmMenu, kTints[1]);
            AppendTintItem(warmMenu, kTints[2]);
            AppendTintItem(warmMenu, kTints[3]);
            AppendTintItem(warmMenu, kTints[4]);
            AppendTintItem(warmMenu, kTints[5]);
            AppendTintItem(warmMenu, kTints[6]);
            AppendTintItem(warmMenu, kTints[7]);

            AppendMenuW(
                menu,
                MF_POPUP,
                reinterpret_cast<UINT_PTR>(warmMenu),
                L"Warm & tinted");
        }

        HMENU paperMenu = CreatePopupMenu();
        if (paperMenu) {
            AppendTintItem(paperMenu, kTints[8]);
            AppendTintItem(paperMenu, kTints[9]);
            AppendTintItem(paperMenu, kTints[10]);
            AppendTintItem(paperMenu, kTints[11]);
            AppendTintItem(paperMenu, kTints[12]);
            AppendTintItem(paperMenu, kTints[13]);
            AppendTintItem(paperMenu, kTints[14]);
            AppendTintItem(paperMenu, kTints[15]);

            AppendMenuW(
                menu,
                MF_POPUP,
                reinterpret_cast<UINT_PTR>(paperMenu),
                L"Paper");
        }

        AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

        AppendMenuW(
            menu,
            MF_STRING,

```

```

        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

POINT pt = {};
GetCursorPos(&pt);

SetForegroundWindow(g_hwnd);

g_activeMenu = menu;
g_menuTracking = true;
g_sliderDragging = false;

g_menuHook = SetWindowsHookExW(
    WH_MSGFILTER,
    MenuMessageFilter,
    nullptr,
    GetCurrentThreadId());

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);
}

```

```

RECT textRc = rc;
textRc.left += 34;
textRc.right -= 8;

DrawTextW(
    dc,
    tint->name,
    -1,
    &textRc,
    DT_SINGLELINE |
    DT_VCENTER |
    DT_LEFT |
    DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWN)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }

                if (FindTint(dis->itemID)) {
                    DrawTintMenuItem(dis);
                    return TRUE;
                }
            }
        }
    }
}

```

```

    }
}
break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;
}

```

```

g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

const DWORD exStyle =
    WS_EX_LAYERED |
    WS_EX_TRANSPARENT |
    WS_EX_NOACTIVATE |
    WS_EX_TOOLWINDOW |
    WS_EX_TOPMOST;

g_hwnd = CreateWindowExW(
    exStyle,
    wc.lpszClassName,
    L"GrainLayer",
    WS_POPUP,
    g_x,
    g_y,
    g_width,
    g_height,
    nullptr,
    nullptr,
    instance,
    nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(
    g_hwnd, 1, MOD_NOREPEAT, VK_F6);

RegisterHotKey(
    g_hwnd, 2, MOD_NOREPEAT, VK_F7);

RegisterHotKey(
    g_hwnd, 3, MOD_NOREPEAT, VK_F8);

RegisterHotKey(
    g_hwnd,
    4,
    MOD_CONTROL |
    MOD_ALT |
    MOD_SHIFT |
    MOD_NOREPEAT,
    'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.15. GrainLayer v1.2.3

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

```

```

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }
}

```

```

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

```

```

        const BYTE gray =
            static_cast<BYTE>(value * 255.0f);

        BYTE r = gray;
        BYTE g = gray;
        BYTE b = gray;

        if (g_hasTint) {
            r = BlendChannel(gray, tintR, TINT_MIX);
            g = BlendChannel(gray, tintG, TINT_MIX);
            b = BlendChannel(gray, tintB, TINT_MIX);
        }

        // Premultiplied alpha required by UpdateLayeredWindow.
        const BYTE a = g_alpha;

        p.r = static_cast<BYTE>(
            (static_cast<unsigned int>(r) * a + 127) / 255);
        p.g = static_cast<BYTE>(
            (static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>(
            (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
}

```

```

    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)

```

```

{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity                %d%%",
        AlphaToPercent());

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

    HBRUSH fillBrush = CreateSolidBrush(
        GetSysColor(COLOR_HIGHLIGHT));
    FillRect(dc, &filled, fillBrush);
    DeleteObject(fillBrush);

    const int radius = 7;
    RECT knob = {
        knobX - radius,
        trackY - radius,
        knobX + radius + 1,
        trackY + radius + 1
    };

    HBRUSH knobBrush = CreateSolidBrush(face);
    FillRect(dc, &knob, knobBrush);
    DeleteObject(knobBrush);

    FrameRect(
        dc,
        &knob,
        static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)

```

```

{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
                case WM_LBUTTONDOWN:
                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        g_sliderDragging = true;
                        SetIntensityFromPoint(g_activeMenu, msg->pt);

                        // Consume the click so the menu stays open.
                        return 1;
                    }
                    break;

                case WM_MOUSEMOVE:
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        return 1;
                    }

                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                    }
                    break;

                case WM_LBUTTONUP:
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        g_sliderDragging = false;

                        // Consume the release so the menu stays open.
                        return 1;
                    }
                    break;
            }
        }
    }
}

```

```

    return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    g_activeMenu = menu;
    g_menuTracking = true;
    g_sliderDragging = false;

    g_menuHook = SetWindowsHookExW(
        WH_MSGFILTER,
        MenuMessageFilter,
        nullptr,
        GetCurrentThreadId());

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,

```

```

    0,
    g_hwnd,
    nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;
}

```

```

HDC dc = GetDC(g_hwnd);
HFONT font = static_cast<HFONT>(
    GetStockObject(DEFAULT_GUI_FONT));

HFONT old = static_cast<HFONT>(
    SelectObject(dc, font));

SIZE size = {};
GetTextExtentPoint32W(
    dc,
    tint->name,
    lstrlenW(tint->name),
    &size);

SelectObject(dc, old);
ReleaseDC(g_hwnd, dc);

mis->itemWidth =
    static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }

                if (FindTint(dis->itemID)) {
                    DrawTintMenuItem(dis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;
        }
    }
}

```

```

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);
}

```

```

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();

    MSG msg = {};
    while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
        TranslateMessage(&msg);
        DispatchMessageW(&msg);
    }

    UnregisterHotKey(g_hwnd, 1);
    UnregisterHotKey(g_hwnd, 2);
    UnregisterHotKey(g_hwnd, 3);
    UnregisterHotKey(g_hwnd, 4);

    RemoveTrayIcon();
    SafeDeleteLayerBitmap();

    return 0;
}

```

10.16. GrainLayer v1.2.4

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <stdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

```

```

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {

```

```

        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);
            p.g = static_cast<BYTE>(
                (static_cast<unsigned int>(g) * a + 127) / 255);
            p.b = static_cast<BYTE>(
                (static_cast<unsigned int>(b) * a + 127) / 255);
            p.a = a;
        }
    }
}

```

```

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WM_APP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {

```

```

        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

```

```

SetBkMode(dc, TRANSPARENT);
SetTextColor(dc, text);

RECT label = rc;
label.left += 8;
label.top += 3;
label.right -= 8;
label.bottom = label.top + 18;

wchar_t textBuffer[64] = {};
swprintf_s(
    textBuffer,
    L"Intensity",
    AlphaToPercent(),
    "%d%%",

DrawTextW(
    dc,
    textBuffer,
    -1,
    &label,
    DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

const int trackLeft = rc.left + 12;
const int trackRight = rc.right - 12;
const int trackY = rc.bottom - 14;
const int trackHeight = 4;

RECT track = {
    trackLeft,
    trackY - trackHeight / 2,
    trackRight,
    trackY + (trackHeight + 1) / 2
};

HBRUSH trackBrush = CreateSolidBrush(shadow);
FillRect(dc, &track, trackBrush);
DeleteObject(trackBrush);

const int percent = AlphaToPercent();
const int knobX =
    trackLeft +
    ((trackRight - trackLeft) * percent) / 100;

RECT filled = track;
filled.right = knobX;

HBRUSH fillBrush = CreateSolidBrush(
    GetSysColor(COLOR_HIGHLIGHT));
FillRect(dc, &filled, fillBrush);
DeleteObject(fillBrush);

const int radius = 7;
RECT knob = {
    knobX - radius,
    trackY - radius,
    knobX + radius + 1,
    trackY + radius + 1
};

HBRUSH knobBrush = CreateSolidBrush(face);
FillRect(dc, &knob, knobBrush);
DeleteObject(knobBrush);

FrameRect(
    dc,
    &knob,
    static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static bool GetIntensityItemRect(HMENU menu, RECT* itemRect)
{
    if (!menu || !itemRect)
        return false;

    const int count = GetMenuItemCount(menu);

    for (int position = 0; position < count; ++position) {
        MENUITEMINFOW mii = {};
        mii.cbSize = sizeof(mii);
        mii.fMask = MIIM_ID;

        if (GetMenuItemInfoW(
            menu,
            static_cast<UINT>(position),
            TRUE,
            &mii) &&
            mii.wID == IDM_INTENSITY_SLIDER) {

```

```

        return GetMenuItemRect(
            g_hwnd,
            menu,
            static_cast<UINT>(position),
            itemRect) != FALSE;
    }
}

return false;
}

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
                case WM_LBUTTONDOWN:
                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        g_sliderDragging = true;
                        SetIntensityFromPoint(g_activeMenu, msg->pt);

                        // Consume the click so the menu stays open.
                        return 1;
                    }
                    break;

                case WM_MOUSEMOVE:
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        return 1;
                    }

                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                    }
                    break;

                case WM_LBUTTONUP:
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        g_sliderDragging = false;

                        // Consume the release so the menu stays open.
                        return 1;
                    }
                    break;
            }
        }
    }
}

```

```

    }

    return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    g_activeMenu = menu;
    g_menuTracking = true;
    g_sliderDragging = false;

    g_menuHook = SetWindowsHookExW(
        WH_MSGFILTER,
        MenuMessageFilter,
        nullptr,
        GetCurrentThreadId());

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,

```

```

        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

```

```

mis->itemHeight = 24;

HDC dc = GetDC(g_hwnd);
HFONT font = static_cast<HFONT>(
    GetStockObject(DEFAULT_GUI_FONT));

HFONT old = static_cast<HFONT>(
    SelectObject(dc, font));

SIZE size = {};
GetTextExtentPoint32W(
    dc,
    tint->name,
    lstrlenW(tint->name),
    &size);

SelectObject(dc, old);
ReleaseDC(g_hwnd, dc);

mis->itemWidth =
    static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNBLCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }

                if (FindTint(dis->itemID)) {
                    DrawTintMenuItem(dis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);

```

```

        return 0;

default:
{
    const TintMenuItem* tint =
        FindTint(LOWORD(wParam));

    if (tint) {
        SetTint(tint->color, true);
        return 0;
    }
}
break;
}
break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
    case 1:
        ToggleOverlay();
        break;

    case 2:
        ChangeOpacity(+5);
        break;

    case 3:
        ChangeOpacity(-5);
        break;

    case 4:
        DestroyWindow(h);
        break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,

```

```

        instance,
        nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(
    g_hwnd, 1, MOD_NOREPEAT, VK_F6);

RegisterHotKey(
    g_hwnd, 2, MOD_NOREPEAT, VK_F7);

RegisterHotKey(
    g_hwnd, 3, MOD_NOREPEAT, VK_F8);

RegisterHotKey(
    g_hwnd,
    4,
    MOD_CONTROL |
    MOD_ALT |
    MOD_SHIFT |
    MOD_NOREPEAT,
    'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.17. GrainLayer v1.2.5

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

```

```

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);

```

```

g_memDC = CreateCompatibleDC(screenDC);

if (!g_memDC) {
    ReleaseDC(nullptr, screenDC);
    return false;
}

g_bitmap = CreateDIBSection(
    screenDC,
    &bi,
    DIB_RGB_COLORS,
    &g_pixels,
    nullptr,
    0);

ReleaseDC(nullptr, screenDC);

if (!g_bitmap || !g_pixels) {
    SafeDeleteLayerBitmap();
    return false;
}

SelectObject(g_memDC, g_bitmap);
return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);
            p.g = static_cast<BYTE>(
                (static_cast<unsigned int>(g) * a + 127) / 255);
            p.b = static_cast<BYTE>(
                (static_cast<unsigned int>(b) * a + 127) / 255);
            p.a = a;
        }
    }
}

```

```

    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WM_APP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

```

```

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

```

```

HBRUSH bgBrush = CreateSolidBrush(bg);
FillRect(dc, &rc, bgBrush);
DeleteObject(bgBrush);

SetBkMode(dc, TRANSPARENT);
SetTextColor(dc, text);

RECT label = rc;
label.left += 8;
label.top += 3;
label.right -= 8;
label.bottom = label.top + 18;

wchar_t textBuffer[64] = {};
swprintf_s(
    textBuffer,
    L"Intensity                %d%%",
    AlphaToPercent());

DrawTextW(
    dc,
    textBuffer,
    -1,
    &label,
    DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

const int trackLeft = rc.left + 12;
const int trackRight = rc.right - 12;
const int trackY = rc.bottom - 14;
const int trackHeight = 4;

RECT track = {
    trackLeft,
    trackY - trackHeight / 2,
    trackRight,
    trackY + (trackHeight + 1) / 2
};

HBRUSH trackBrush = CreateSolidBrush(shadow);
FillRect(dc, &track, trackBrush);
DeleteObject(trackBrush);

const int percent = AlphaToPercent();
const int knobX =
    trackLeft +
    ((trackRight - trackLeft) * percent) / 100;

RECT filled = track;
filled.right = knobX;

HBRUSH fillBrush = CreateSolidBrush(
    GetSysColor(COLOR_HIGHLIGHT));
FillRect(dc, &filled, fillBrush);
DeleteObject(fillBrush);

const int radius = 7;
RECT knob = {
    knobX - radius,
    trackY - radius,
    knobX + radius + 1,
    trackY + radius + 1
};

HBRUSH knobBrush = CreateSolidBrush(face);
FillRect(dc, &knob, knobBrush);
DeleteObject(knobBrush);

FrameRect(
    dc,
    &knob,
    static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static bool GetIntensityItemRect(HMENU menu, RECT* itemRect)
{
    if (!menu || !itemRect)
        return false;

    const int count = GetMenuItemCount(menu);

    for (int position = 0; position < count; ++position) {
        MENUITEMINFOW mii = {};
        mii.cbSize = sizeof(mii);
        mii.fMask = MIIM_ID;

        if (GetMenuItemInfoW(
            menu,
            static_cast<UINT>(position),

```

```

        TRUE,
        &mii) &&
        mii.wID == IDM_INTENSITY_SLIDER) {

        return GetMenuItemRect(
            g_hwnd,
            menu,
            static_cast<UINT>(position),
            itemRect) != FALSE;
    }
}

return false;
}

static HWND FindPopupMenuWindow()
{
    // Standard Win32 popup menus are implemented by the internal #32768
    // window class. This is used only to force an immediate repaint of the
    // owner-drawn slider while TrackPopupMenu is running.
    return FindWindowExW(
        nullptr,
        nullptr,
        L"#32768",
        nullptr);
}

static void RefreshPopupMenu()
{
    HWND menuWindow = FindPopupMenuWindow();
    if (menuWindow) {
        InvalidateRect(menuWindow, nullptr, FALSE);
        UpdateWindow(menuWindow);
    }
}

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha) {
        PresentLayer();
        RefreshPopupMenu();
    }
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
            case WM_LBUTTONDOWN:
                if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                    g_sliderDragging = true;

                    // Keep receiving mouse movement even when the pointer

```

```

        // leaves the slider while the button is held.
        HWND menuWindow = FindPopupMenuWindow();
        if (menuWindow)
            SetCapture(menuWindow);

        SetIntensityFromPoint(g_activeMenu, msg->pt);

        // Consume the click so the menu stays open.
        return 1;
    }
    break;

case WM_MOUSEMOVE:
    if (g_sliderDragging) {
        SetIntensityFromPoint(g_activeMenu, msg->pt);
        return 1;
    }

    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
        SetIntensityFromPoint(g_activeMenu, msg->pt);
    }
    break;

case WM_LBUTTONDOWN:
    if (g_sliderDragging) {
        SetIntensityFromPoint(g_activeMenu, msg->pt);
        g_sliderDragging = false;

        if (GetCapture() == FindPopupMenuWindow())
            ReleaseCapture();

        // Consume the release so the menu stays open.
        return 1;
    }
    break;
}
}
}

return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
    }
}

```

```

        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    g_activeMenu = menu;
    g_menuTracking = true;
    g_sliderDragging = false;

    g_menuHook = SetWindowsHookExW(
        WH_MSGFILTER,
        MenuMessageFilter,
        nullptr,
        GetCurrentThreadId());

    TrackPopupMenu(
        menu,
        TPM_RIGHTBUTTON |
        TPM_BOTTOMALIGN |
        TPM_LEFTALIGN,
        pt.x,
        pt.y,
        0,
        g_hwnd,
        nullptr);

    if (g_menuHook) {
        UnhookWindowsHookEx(g_menuHook);
        g_menuHook = nullptr;
    }

    g_sliderDragging = false;
    g_menuTracking = false;
    g_activeMenu = nullptr;

    DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

```

```

const COLORREF text =
    selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

HBRUSH bg = CreateSolidBrush(background);
FillRect(dc, &rc, bg);
DeleteObject(bg);

DrawTintSwatch(dc, rc, tint->color);

SetBkMode(dc, TRANSPARENT);
SetTextColor(dc, text);

RECT textRc = rc;
textRc.left += 34;
textRc.right -= 8;

DrawTextW(
    dc,
    tint->name,
    -1,
    &textRc,
    DT_SINGLELINE |
    DT_VCENTER |
    DT_LEFT |
    DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT*>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT*>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWN)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:

```

```

        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }

                if (FindTint(dis->itemID)) {
                    DrawTintMenuItem(dis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
            case IDM_TOGGLE:
                ToggleOverlay();
                return 0;

            case IDM_OPACITY_UP:
                ChangeOpacity(+5);
                return 0;

            case IDM_OPACITY_DOWN:
                ChangeOpacity(-5);
                return 0;

            case IDM_EXIT:
                DestroyWindow(h);
                return 0;

            case IDM_TINT_NONE:
                SetTint(RGB(255, 255, 255), false);
                return 0;

            default:
            {
                const TintMenuItem* tint =
                    FindTint(LOWORD(wParam));

                if (tint) {
                    SetTint(tint->color, true);
                    return 0;
                }
            }
        }
        break;
    }
    break;

    case WM_NCHITTEST:
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();
        return 0;

    case WM_HOTKEY:
        switch (wParam) {
            case 1:
                ToggleOverlay();
                break;

            case 2:
                ChangeOpacity(+5);
                break;

            case 3:
                ChangeOpacity(-5);
                break;

            case 4:
                DestroyWindow(h);
                break;
        }
        return 0;

    case WM_DESTROY:
        PostQuitMessage(0);
        return 0;
    }

    return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{

```

```

INITCOMMONCONTROLSEX icc = {};
icc.dwSize = sizeof(icc);
icc.dwICC = ICC_BAR_CLASSES;
InitCommonControlsEx(&icc);

WNDCLASSEXW wc = {};
wc.cbSize = sizeof(wc);
wc.hInstance = instance;
wc.lpfnWndProc = WndProc;
wc.lpszClassName = L"GrainLayerOverlay";
wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

if (!RegisterClassExW(&wc))
    return 1;

g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

const DWORD exStyle =
    WS_EX_LAYERED |
    WS_EX_TRANSPARENT |
    WS_EX_NOACTIVATE |
    WS_EX_TOOLWINDOW |
    WS_EX_TOPMOST;

g_hwnd = CreateWindowExW(
    exStyle,
    wc.lpszClassName,
    L"GrainLayer",
    WS_POPUP,
    g_x,
    g_y,
    g_width,
    g_height,
    nullptr,
    nullptr,
    instance,
    nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(
    g_hwnd, 1, MOD_NOREPEAT, VK_F6);

RegisterHotKey(
    g_hwnd, 2, MOD_NOREPEAT, VK_F7);

RegisterHotKey(
    g_hwnd, 3, MOD_NOREPEAT, VK_F8);

RegisterHotKey(
    g_hwnd,
    4,
    MOD_CONTROL |
    MOD_ALT |
    MOD_SHIFT |
    MOD_NOREPEAT,
    'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.18. GrainLayer v1.2.6

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>

```

```

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
}

```

```

    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

```

```

    }

    const int tx = x & (TILE - 1);
    const int ty = y & (TILE - 1);

    const uint32_t h =
        Hash(static_cast<uint32_t>(tx) * 374761393u ^
            static_cast<uint32_t>(ty) * 668265263u);

    const float noise =
        static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

    float value = 0.50f + noise * 0.10f;
    value = std::clamp(value, 0.25f, 0.75f);

    const BYTE gray =
        static_cast<BYTE>(value * 255.0f);

    BYTE r = gray;
    BYTE g = gray;
    BYTE b = gray;

    if (g_hasTint) {
        r = BlendChannel(gray, tintR, TINT_MIX);
        g = BlendChannel(gray, tintG, TINT_MIX);
        b = BlendChannel(gray, tintB, TINT_MIX);
    }

    // Premultiplied alpha required by UpdateLayeredWindow.
    const BYTE a = g_alpha;

    p.r = static_cast<BYTE>(
        (static_cast<unsigned int>(r) * a + 127) / 255);
    p.g = static_cast<BYTE>(
        (static_cast<unsigned int>(g) * a + 127) / 255);
    p.b = static_cast<BYTE>(
        (static_cast<unsigned int>(b) * a + 127) / 255);
    p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{

```

```

        g_enabled = !g_enabled;
        PresentLayer();
    }

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(nullptr, IDI_APPLICATION);

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =

```

```

        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity AlphaToPercent()",
        %d%%",

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

    HBRUSH fillBrush = CreateSolidBrush(
        GetSysColor(COLOR_HIGHLIGHT));
    FillRect(dc, &filled, fillBrush);
    DeleteObject(fillBrush);

    const int radius = 7;
    RECT knob = {
        knobX - radius,
        trackY - radius,

```

```

        knobX + radius + 1,
        trackY + radius + 1
    };

    HBRUSH knobBrush = CreateSolidBrush(face);
    FillRect(dc, &knob, knobBrush);
    DeleteObject(knobBrush);

    FrameRect(
        dc,
        &knob,
        static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static bool GetIntensityItemRect(HMENU menu, RECT* itemRect)
{
    if (!menu || !itemRect)
        return false;

    const int count = GetMenuItemCount(menu);

    for (int position = 0; position < count; ++position) {
        MENUITEMINFOW mii = {};
        mii.cbSize = sizeof(mii);
        mii.fMask = MIIM_ID;

        if (GetMenuItemInfoW(
            menu,
            static_cast<UINT>(position),
            TRUE,
            &mii) &&
            mii.wID == IDM_INTENSITY_SLIDER) {
            return GetMenuItemRect(
                g_hwnd,
                menu,
                static_cast<UINT>(position),
                itemRect) != FALSE;
        }
    }

    return false;
}

static HWND FindPopupMenuWindow()
{
    // Standard Win32 popup menus are implemented by the internal #32768
    // window class. This is used only to force an immediate repaint of the
    // owner-drawn slider while TrackPopupMenu is running.
    return FindWindowExW(
        nullptr,
        nullptr,
        L"#32768",
        nullptr);
}

static void RefreshPopupMenu()
{
    HWND menuWindow = FindPopupMenuWindow();
    if (menuWindow) {
        InvalidateRect(menuWindow, nullptr, FALSE);
        UpdateWindow(menuWindow);
    }
}

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /

```

```

        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha) {
        PresentLayer();
        RefreshPopupMenu();
    }
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
                case WM_LBUTTONDOWN:
                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        g_sliderDragging = true;

                        // Keep receiving mouse movement even when the pointer
                        // leaves the slider while the button is held.
                        HWND menuWindow = FindPopupMenuWindow();
                        if (menuWindow)
                            SetCapture(menuWindow);

                        SetIntensityFromPoint(g_activeMenu, msg->pt);

                        // Consume the click so the menu stays open.
                        return 1;
                    }
                    break;

                case WM_MOUSEMOVE:
                    // A plain mouseover must not change the intensity.
                    // Only a left-button drag updates the slider.
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        return 1;
                    }
                    break;

                case WM_LBUTTONUP:
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        g_sliderDragging = false;

                        if (GetCapture() == FindPopupMenuWindow())
                            ReleaseCapture();

                        // Consume the release so the menu stays open.
                        return 1;
                    }
                    break;
            }
        }
    }

    return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(

```

```

        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

AppendIntensitySlider(menu);

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

HMENU warmMenu = CreatePopupMenu();
if (warmMenu) {
    AppendTintItem(warmMenu, kTints[1]);
    AppendTintItem(warmMenu, kTints[2]);
    AppendTintItem(warmMenu, kTints[3]);
    AppendTintItem(warmMenu, kTints[4]);
    AppendTintItem(warmMenu, kTints[5]);
    AppendTintItem(warmMenu, kTints[6]);
    AppendTintItem(warmMenu, kTints[7]);

    AppendMenuW(
        menu,
        MF_POPUP,
        reinterpret_cast<UINT_PTR>(warmMenu),
        L"Warm & tinted");
}

HMENU paperMenu = CreatePopupMenu();
if (paperMenu) {
    AppendTintItem(paperMenu, kTints[8]);
    AppendTintItem(paperMenu, kTints[9]);
    AppendTintItem(paperMenu, kTints[10]);
    AppendTintItem(paperMenu, kTints[11]);
    AppendTintItem(paperMenu, kTints[12]);
    AppendTintItem(paperMenu, kTints[13]);
    AppendTintItem(paperMenu, kTints[14]);
    AppendTintItem(paperMenu, kTints[15]);

    AppendMenuW(
        menu,
        MF_POPUP,
        reinterpret_cast<UINT_PTR>(paperMenu),
        L"Paper");
}

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(
    menu,
    MF_STRING,
    IDM_EXIT,
    L"Exit (Ctrl+Alt+Shift+Q)");

POINT pt = {};
GetCursorPos(&pt);

SetForegroundWindow(g_hwnd);

g_activeMenu = menu;
g_menuTracking = true;
g_sliderDragging = false;

g_menuHook = SetWindowsHookExW(
    WH_MSGFILTER,
    MenuMessageFilter,
    nullptr,
    GetCurrentThreadId());

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

```

```

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
            ? GetSysColor(COLOR_HIGHLIGHT)
            : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
            ? GetSysColor(COLOR_HIGHLIGHTTEXT)
            : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(

```

```

HWND h,
UINT msg,
WPARAM wParam,
LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONUP)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }

                if (FindTint(dis->itemID)) {
                    DrawTintMenuItem(dis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
        }
        break;

    case WM_NCHITTEST:
        return HTTRANSPARENT;

    case WM_MOUSEACTIVATE:
        return MA_NOACTIVATE;

    case WM_DISPLAYCHANGE:
        Refresh();

```

```

        return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |

```

```

        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.19. GrainLayer v1.3

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>
#include "resource.h"

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
}

```

```

};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }
}

```

```

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =
                static_cast<BYTE>(value * 255.0f);

            BYTE r = gray;
            BYTE g = gray;
            BYTE b = gray;

            if (g_hasTint) {
                r = BlendChannel(gray, tintR, TINT_MIX);
                g = BlendChannel(gray, tintG, TINT_MIX);
                b = BlendChannel(gray, tintB, TINT_MIX);
            }

            // Premultiplied alpha required by UpdateLayeredWindow.
            const BYTE a = g_alpha;

            p.r = static_cast<BYTE>(
                (static_cast<unsigned int>(r) * a + 127) / 255);
            p.g = static_cast<BYTE>(
                (static_cast<unsigned int>(g) * a + 127) / 255);
            p.b = static_cast<BYTE>(
                (static_cast<unsigned int>(b) * a + 127) / 255);
            p.a = a;
        }
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(

```

```

        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hwnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WM_APP_TRAY;
    g_nid.hIcon = LoadIconW(GetModuleHandleW(nullptr), MAKEINTRESOURCEW(IDI_APP_ICON));

    wcsncpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),

```

```

        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity                %d%%",
        AlphaToPercent());

    DrawTextW(
        dc,
        textBuffer,

```

```

        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

const int trackLeft = rc.left + 12;
const int trackRight = rc.right - 12;
const int trackY = rc.bottom - 14;
const int trackHeight = 4;

RECT track = {
    trackLeft,
    trackY - trackHeight / 2,
    trackRight,
    trackY + (trackHeight + 1) / 2
};

HBRUSH trackBrush = CreateSolidBrush(shadow);
FillRect(dc, &track, trackBrush);
DeleteObject(trackBrush);

const int percent = AlphaToPercent();
const int knobX =
    trackLeft +
    ((trackRight - trackLeft) * percent) / 100;

RECT filled = track;
filled.right = knobX;

HBRUSH fillBrush = CreateSolidBrush(
    GetSysColor(COLOR_HIGHLIGHT));
FillRect(dc, &filled, fillBrush);
DeleteObject(fillBrush);

const int radius = 7;
RECT knob = {
    knobX - radius,
    trackY - radius,
    knobX + radius + 1,
    trackY + radius + 1
};

HBRUSH knobBrush = CreateSolidBrush(face);
FillRect(dc, &knob, knobBrush);
DeleteObject(knobBrush);

FrameRect(
    dc,
    &knob,
    static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static bool GetIntensityItemRect(HMENU menu, RECT* itemRect)
{
    if (!menu || !itemRect)
        return false;

    const int count = GetMenuItemCount(menu);

    for (int position = 0; position < count; ++position) {
        MENUITEMINFOW mii = {};
        mii.cbSize = sizeof(mii);
        mii.fMask = MIIM_ID;

        if (GetMenuItemInfoW(
            menu,
            static_cast<UINT>(position),
            TRUE,
            &mii) &&
            mii.wID == IDM_INTENSITY_SLIDER) {
            return GetMenuItemRect(
                g_hwnd,
                menu,
                static_cast<UINT>(position),
                itemRect) != FALSE;
        }
    }

    return false;
}

static HWND FindPopupMenuWindow()
{
    // Standard Win32 popup menus are implemented by the internal #32768
    // window class. This is used only to force an immediate repaint of the
    // owner-drawn slider while TrackPopupMenu is running.
    return FindWindowExW(
        nullptr,

```

```

        nullptr,
        L"#32768",
        nullptr);
}

static void RefreshPopupMenu()
{
    HWND menuWindow = FindPopupMenuWindow();
    if (menuWindow) {
        InvalidateRect(menuWindow, nullptr, FALSE);
        UpdateWindow(menuWindow);
    }
}

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha) {
        PresentLayer();
        RefreshPopupMenu();
    }
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
                case WM_LBUTTONDOWN:
                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        g_sliderDragging = true;

                        // Keep receiving mouse movement even when the pointer
                        // leaves the slider while the button is held.
                        HWND menuWindow = FindPopupMenuWindow();
                        if (menuWindow)
                            SetCapture(menuWindow);

                        SetIntensityFromPoint(g_activeMenu, msg->pt);

                        // Consume the click so the menu stays open.
                        return 1;
                    }
                    break;

                case WM_MOUSEMOVE:
                    // A plain mouseover must not change the intensity.
                    // Only a left-button drag updates the slider.
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        return 1;
                    }
                    break;

                case WM_LBUTTONUP:

```

```

        if (g_sliderDragging) {
            SetIntensityFromPoint(g_activeMenu, msg->pt);
            g_sliderDragging = false;

            if (GetCapture() == FindPopupMenuWindow())
                ReleaseCapture();

            // Consume the release so the menu stays open.
            return 1;
        }
        break;
    }
}

return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Disable overlay (F6)" : L"Enable overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_EXIT,
        L"Exit (Ctrl+Alt+Shift+Q)");

    POINT pt = {};
    GetCursorPos(&pt);

    SetForegroundWindow(g_hwnd);

    g_activeMenu = menu;
    g_menuTracking = true;
}

```

```

g_sliderDragging = false;

g_menuHook = SetWindowsHookExW(
    WH_MSGFILTER,
    MenuMessageFilter,
    nullptr,
    GetCurrentThreadId());

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |

```

```

        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }

                if (FindTint(dis->itemID)) {
                    DrawTintMenuItem(dis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_COMMAND:
        switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;
        }
    }
}

```

```

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wc.hIcon = LoadIconW(instance, MAKEINTRESOURCEW(IDI_APP_ICON));
    wc.hIconSm = LoadIconW(instance, MAKEINTRESOURCEW(IDI_APP_ICON));

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |

```

```

        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

g_hwnd = CreateWindowExW(
    exStyle,
    wc.lpszClassName,
    L"GrainLayer",
    WS_POPUP,
    g_x,
    g_y,
    g_width,
    g_height,
    nullptr,
    nullptr,
    instance,
    nullptr);

if (!g_hwnd)
    return 2;

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(
    g_hwnd, 1, MOD_NOREPEAT, VK_F6);

RegisterHotKey(
    g_hwnd, 2, MOD_NOREPEAT, VK_F7);

RegisterHotKey(
    g_hwnd, 3, MOD_NOREPEAT, VK_F8);

RegisterHotKey(
    g_hwnd,
    4,
    MOD_CONTROL |
    MOD_ALT |
    MOD_SHIFT |
    MOD_NOREPEAT,
    'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.20. GrainLayer v1.4

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>
#include <string>
#include <shlobj.h>
#include "resource.h"

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")
#pragma comment(lib, "advapi32.lib")
#pragma comment(lib, "ole32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_UP = 1002;
constexpr UINT IDM_OPACITY_DOWN = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
constexpr UINT IDM_STARTUP = 1007;
static bool g_menuTracking = false;

constexpr int INTENSITY_POPUP_W = 260;

```

```

constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)
{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

```

```

}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {
        for (int x = 0; x < g_width; ++x) {
            Pixel& p = pixels[y * g_width + x];

            if (!g_enabled || g_alpha == 0) {
                p = { 0, 0, 0, 0 };
                continue;
            }

            const int tx = x & (TILE - 1);
            const int ty = y & (TILE - 1);

            const uint32_t h =
                Hash(static_cast<uint32_t>(tx) * 374761393u ^
                    static_cast<uint32_t>(ty) * 668265263u);

            const float noise =
                static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

            float value = 0.50f + noise * 0.10f;
            value = std::clamp(value, 0.25f, 0.75f);

            const BYTE gray =

```

```

        static_cast<BYTE>(value * 255.0f);

    BYTE r = gray;
    BYTE g = gray;
    BYTE b = gray;

    if (g_hasTint) {
        r = BlendChannel(gray, tintR, TINT_MIX);
        g = BlendChannel(gray, tintG, TINT_MIX);
        b = BlendChannel(gray, tintB, TINT_MIX);
    }

    // Premultiplied alpha required by UpdateLayeredWindow.
    const BYTE a = g_alpha;

    p.r = static_cast<BYTE>((static_cast<unsigned int>(r) * a + 127) / 255);
    p.g = static_cast<BYTE>((static_cast<unsigned int>(g) * a + 127) / 255);
    p.b = static_cast<BYTE>((static_cast<unsigned int>(b) * a + 127) / 255);
    p.a = a;
}
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;

    PositionWindow();
    PresentLayer();
}

static int AlphaToPercent();
static void PercentToAlpha(int percent);

static std::wstring GetSettingsPath()
{
    PWSTR appData = nullptr;
    std::wstring path;

    if (SUCCEEDED(SHGetKnownFolderPath(
        FOLDERID_RoamingAppData, 0, nullptr, &appData)) && appData) {
        path = appData;
        CoTaskMemFree(appData);

        path += L"\\GrainLayer";
        CreateDirectoryW(path.c_str(), nullptr);
        path += L"\\settings.ini";
    }
}

```

```

    return path;
}

static UINT GetSelectedTintId()
{
    if (!g_hasTint)
        return IDM_TINT_NONE;

    for (const auto& tint : kTints) {
        if (tint.color == g_tint)
            return tint.id;
    }

    return IDM_TINT_NONE;
}

static void SaveSettings()
{
    const std::wstring path = GetSettingsPath();
    if (path.empty())
        return;

    wchar_t buffer[32] = {};
    swprintf_s(buffer, L"%d", AlphaToPercent());
    WritePrivateProfileStringW(L"GrainLayer", L"Intensity", buffer, path.c_str());

    WritePrivateProfileStringW(
        L"GrainLayer", L"Enabled", g_enabled ? L"1" : L"0", path.c_str());

    swprintf_s(buffer, L"%u", GetSelectedTintId());
    WritePrivateProfileStringW(L"GrainLayer", L"Tint", buffer, path.c_str());
}

static void LoadSettings()
{
    const std::wstring path = GetSettingsPath();
    if (path.empty())
        return;

    wchar_t buffer[32] = {};

    GetPrivateProfileStringW(
        L"GrainLayer", L"Intensity", L"25", buffer, _countof(buffer), path.c_str());
    PercentToAlpha(_wtoi(buffer));

    GetPrivateProfileStringW(
        L"GrainLayer", L"Enabled", L"1", buffer, _countof(buffer), path.c_str());
    g_enabled = (_wtoi(buffer) != 0);

    GetPrivateProfileStringW(
        L"GrainLayer", L"Tint", L"1100", buffer, _countof(buffer), path.c_str());
    const UINT tintId = static_cast<UINT>(_wtoi(buffer));

    if (tintId == IDM_TINT_NONE) {
        g_tint = RGB(255, 255, 255);
        g_hasTint = false;
    } else {
        const TintMenuItem* tint = nullptr;
        for (const auto& item : kTints) {
            if (item.id == tintId) {
                tint = &item;
                break;
            }
        }

        if (tint) {
            g_tint = tint->color;
            g_hasTint = true;
        }
    }
}

static bool IsStartupEnabled()
{
    HKEY key = nullptr;
    if (RegOpenKeyExW(
        HKEY_CURRENT_USER,
        L"Software\\Microsoft\\Windows\\CurrentVersion\\Run",
        0,
        KEY_QUERY_VALUE,
        &key) != ERROR_SUCCESS)
        return false;

    wchar_t value[1024] = {};
    DWORD type = 0;
    DWORD size = sizeof(value);
    const LONG result = RegQueryValueExW(
        key, L"GrainLayer", nullptr, &type,
        reinterpret_cast<LPBYTE>(value), &size);
    RegCloseKey(key);

    return result == ERROR_SUCCESS &&
        (type == REG_SZ || type == REG_EXPAND_SZ) &&
        value[0] != L'\0';
}

static bool SetStartupEnabled(bool enabled)
{

```

```

HKEY key = nullptr;
const LONG openResult = RegOpenKeyExW(
    HKEY_CURRENT_USER,
    L"Software\\Microsoft\\Windows\\CurrentVersion\\Run",
    0,
    KEY_SET_VALUE,
    &key);

if (openResult != ERROR_SUCCESS)
    return false;

LONG result = ERROR_SUCCESS;

if (enabled) {
    wchar_t exePath[MAX_PATH] = {};
    if (GetModuleFileNameW(nullptr, exePath, _countof(exePath)) == 0) {
        result = GetLastError();
    } else {
        std::wstring command = L"";
        command += exePath;
        command += L"\\";

        result = RegSetValueExW(
            key,
            L"GrainLayer",
            0,
            REG_SZ,
            reinterpret_cast<const BYTE*>(command.c_str()),
            static_cast<DWORD>((command.size() + 1) * sizeof(wchar_t)));
    }
} else {
    result = RegDeleteValueW(key, L"GrainLayer");
    if (result == ERROR_FILE_NOT_FOUND)
        result = ERROR_SUCCESS;
}

RegCloseKey(key);
return result == ERROR_SUCCESS;
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
    SaveSettings();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();
    SaveSettings();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
    SaveSettings();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
    g_nid.hIcon = LoadIconW(GetModuleHandleW(nullptr), MAKEINTRESOURCEW(IDI_APP_ICON));

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;

```

```

        mii.wID = tint.id;
        mii.fType = MFT_OWNERDRAW;
        mii.dwItemData =
            reinterpret_cast<ULONG_PTR>(&tint);

        InsertMenuItemW(
            menu,
            GetMenuItemCount(menu),
            TRUE,
            &mii);
    }

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};

```

```

    swprintf_s(
        textBuffer,
        L"Intensity          %d%%",
        AlphaToPercent());

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

    HBRUSH fillBrush = CreateSolidBrush(
        GetSysColor(COLOR_HIGHLIGHT));
    FillRect(dc, &filled, fillBrush);
    DeleteObject(fillBrush);

    const int radius = 7;
    RECT knob = {
        knobX - radius,
        trackY - radius,
        knobX + radius + 1,
        trackY + radius + 1
    };

    HBRUSH knobBrush = CreateSolidBrush(face);
    FillRect(dc, &knob, knobBrush);
    DeleteObject(knobBrush);

    FrameRect(
        dc,
        &knob,
        static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static bool GetIntensityItemRect(HMENU menu, RECT* itemRect)
{
    if (!menu || !itemRect)
        return false;

    const int count = GetMenuItemCount(menu);

    for (int position = 0; position < count; ++position) {
        MENUITEMINFOW mii = {};
        mii.cbSize = sizeof(mii);
        mii.fMask = MIIM_ID;

        if (GetMenuItemInfoW(
            menu,
            static_cast<UINT>(position),
            TRUE,
            &mii) &&
            mii.wID == IDM_INTENSITY_SLIDER) {
            return GetMenuItemRect(
                g_hwnd,
                menu,
                static_cast<UINT>(position),
                itemRect) != FALSE;
        }
    }

    return false;
}

```

```

static HWND FindPopupMenuWindow()
{
    // Standard Win32 popup menus are implemented by the internal #32768
    // window class. This is used only to force an immediate repaint of the
    // owner-drawn slider while TrackPopupMenu is running.
    return FindWindowExW(
        nullptr,
        nullptr,
        L"#32768",
        nullptr);
}

static void RefreshPopupMenu()
{
    HWND menuWindow = FindPopupMenuWindow();
    if (menuWindow) {
        InvalidateRect(menuWindow, nullptr, FALSE);
        UpdateWindow(menuWindow);
    }
}

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha) {
        PresentLayer();
        RefreshPopupMenu();
        SaveSettings();
    }
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,
    LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
            case WM_LBUTTONDOWN:
                if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                    g_sliderDragging = true;

                    // Keep receiving mouse movement even when the pointer
                    // leaves the slider while the button is held.
                    HWND menuWindow = FindPopupMenuWindow();
                    if (menuWindow)
                        SetCapture(menuWindow);

                    SetIntensityFromPoint(g_activeMenu, msg->pt);

                    // Consume the click so the menu stays open.
                    return 1;
                }
                break;

            case WM_MOUSEMOVE:
                // A plain mouseover must not change the intensity.

```

```

        // Only a left-button drag updates the slider.
        if (g_sliderDragging) {
            SetIntensityFromPoint(g_activeMenu, msg->pt);
            return 1;
        }
        break;

    case WM_LBUTTONDOWN:
        if (g_sliderDragging) {
            SetIntensityFromPoint(g_activeMenu, msg->pt);
            g_sliderDragging = false;

            if (GetCapture() == FindPopupMenuWindow())
                ReleaseCapture();

            // Consume the release so the menu stays open.
            return 1;
        }
        break;
    }
}

return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Toggle overlay (F6)" : L"Toggle overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }

    HMENU paperMenu = CreatePopupMenu();
    if (paperMenu) {
        AppendTintItem(paperMenu, kTints[8]);
        AppendTintItem(paperMenu, kTints[9]);
        AppendTintItem(paperMenu, kTints[10]);
        AppendTintItem(paperMenu, kTints[11]);
        AppendTintItem(paperMenu, kTints[12]);
        AppendTintItem(paperMenu, kTints[13]);
        AppendTintItem(paperMenu, kTints[14]);
        AppendTintItem(paperMenu, kTints[15]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(paperMenu),
            L"Paper");
    }

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(
        menu,
        MF_STRING | (IsStartupEnabled() ? MF_CHECKED : 0),
        IDM_STARTUP,
        L"Open at startup");
}

```

```

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(
    menu,
    MF_STRING,
    IDM_EXIT,
    L"Exit (Ctrl+Alt+Shift+Q)");

POINT pt = {};
GetCursorPos(&pt);

SetForegroundWindow(g_hwnd);

g_activeMenu = menu;
g_menuTracking = true;
g_sliderDragging = false;

g_menuHook = SetWindowsHookExW(
    WH_MSGFILTER,
    MenuMessageFilter,
    nullptr,
    GetCurrentThreadId());

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);
}

```

```

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNBLCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

            if (mis && mis->CtlType == ODT_MENU) {
                auto* mutableMis =
                    const_cast<MEASUREITEMSTRUCT*>(mis);

                if (mis->itemID == IDM_INTENSITY_SLIDER) {
                    MeasureIntensitySlider(mutableMis);
                    return TRUE;
                }

                if (FindTint(mis->itemID)) {
                    MeasureTintMenuItem(mutableMis);
                    return TRUE;
                }
            }
        }
        break;

    case WM_DRAWITEM:
        if (wParam == 0) {
            const auto* dis =
                reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

            if (dis && dis->CtlType == ODT_MENU) {
                if (dis->itemID == IDM_INTENSITY_SLIDER) {
                    DrawIntensitySlider(dis);
                    return TRUE;
                }
            }
        }
    }
}

```

```

        if (FindTint(dis->itemID)) {
            DrawTintMenuItem(dis);
            return TRUE;
        }
    }
}
break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_STARTUP:
            SetStartupEnabled(!IsStartupEnabled());
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
        break;
    }
    break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(+5);
            break;

        case 3:
            ChangeOpacity(-5);
            break;

        case 4:
            DestroyWindow(h);
            break;
    }
    return 0;

case WM_DESTROY:
    SaveSettings();
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

```

```

WNDCLASSEXW wc = {};
wc.cbSize = sizeof(wc);
wc.hInstance = instance;
wc.lpszClassName = L"GrainLayerOverlay";
wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
wc.hIcon = LoadIconW(instance, MAKEINTRESOURCEW(IDI_APP_ICON));
wc.hIconSm = LoadIconW(instance, MAKEINTRESOURCEW(IDI_APP_ICON));

if (!RegisterClassExW(&wc))
    return 1;

g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

const DWORD exStyle =
    WS_EX_LAYERED |
    WS_EX_TRANSPARENT |
    WS_EX_NOACTIVATE |
    WS_EX_TOOLWINDOW |
    WS_EX_TOPMOST;

g_hwnd = CreateWindowExW(
    exStyle,
    wc.lpszClassName,
    L"GrainLayer",
    WS_POPUP,
    g_x,
    g_y,
    g_width,
    g_height,
    nullptr,
    nullptr,
    instance,
    nullptr);

if (!g_hwnd)
    return 2;

LoadSettings();

if (!CreateLayerBitmap())
    return 3;

AddTrayIcon();

RegisterHotKey(
    g_hwnd, 1, MOD_NOREPEAT, VK_F6);

RegisterHotKey(
    g_hwnd, 2, MOD_NOREPEAT, VK_F7);

RegisterHotKey(
    g_hwnd, 3, MOD_NOREPEAT, VK_F8);

RegisterHotKey(
    g_hwnd,
    4,
    MOD_CONTROL |
    MOD_ALT |
    MOD_SHIFT |
    MOD_NOREPEAT,
    'Q');

ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
PositionWindow();
PresentLayer();

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

10.21. GrainLayer v1.4.1

```

#include <windows.h>
#include <shellapi.h>
#include <commctrl.h>
#include <algorithm>
#include <cstdint>
#include <vector>
#include <string>
#include <shlobj.h>

```

```

#include "resource.h"

#pragma comment(lib, "user32.lib")
#pragma comment(lib, "gdi32.lib")
#pragma comment(lib, "shell32.lib")
#pragma comment(lib, "comctl32.lib")
#pragma comment(lib, "advapi32.lib")
#pragma comment(lib, "ole32.lib")

constexpr UINT WMAPP_TRAY = WM_APP + 1;

constexpr UINT IDM_TOGGLE = 1001;
constexpr UINT IDM_OPACITY_DOWN = 1002;
constexpr UINT IDM_OPACITY_UP = 1003;
constexpr UINT IDM_EXIT = 1004;
constexpr UINT IDM_INTENSITY = 1005;
constexpr UINT IDM_INTENSITY_SLIDER = 1006;
constexpr UINT IDM_STARTUP = 1007;
static bool g_menuTracking = false;
static void RefreshPopupMenu();

constexpr int INTENSITY_POPUP_W = 260;
constexpr int INTENSITY_POPUP_H = 86;

static HWND g_intensityPopup = nullptr;
static HWND g_intensitySlider = nullptr;
static HWND g_intensityLabel = nullptr;

constexpr UINT IDM_TINT_NONE = 1100;
constexpr UINT IDM_TINT_FOXED_AMBER = 1101;
constexpr UINT IDM_TINT_BOOKCLOTH = 1102;
constexpr UINT IDM_TINT_RECYCLED_KRAFT = 1103;
constexpr UINT IDM_TINT_PLUM_KOZO = 1104;
constexpr UINT IDM_TINT_ROSE_QUARTZ = 1105;
constexpr UINT IDM_TINT_SAGE_PRESS = 1106;
constexpr UINT IDM_TINT_NORDIC_SKY = 1107;
constexpr UINT IDM_TINT_SOFT_WAVE = 1201;
constexpr UINT IDM_TINT_RICE_PAPER = 1202;
constexpr UINT IDM_TINT_LAID_COTTON = 1203;
constexpr UINT IDM_TINT_NEWSPRINT = 1204;
constexpr UINT IDM_TINT_COLD_PRESS = 1205;
constexpr UINT IDM_TINT_ARTIST_CANVAS = 1206;
constexpr UINT IDM_TINT_FELT_SIDE = 1207;
constexpr UINT IDM_TINT_FROST_GLASSINE = 1208;

static NOTIFYICONDATAW g_nid = {};
static HWND g_hwnd = nullptr;

static HBITMAP g_bitmap = nullptr;
static HDC g_memDC = nullptr;
static void* g_pixels = nullptr;

static int g_x = 0;
static int g_y = 0;
static int g_width = 0;
static int g_height = 0;

static BYTE g_alpha = 34;
static bool g_enabled = true;

// COLORREF is used only as the selected tint definition.
// The actual tint is deliberately mixed very weakly into the paper texture.
static COLORREF g_tint = RGB(255, 255, 255);
static bool g_hasTint = false;

struct Pixel {
    BYTE b, g, r, a;
};

struct TintMenuItem {
    UINT id;
    const wchar_t* name;
    COLORREF color;
};

static const TintMenuItem kTints[] = {
    { IDM_TINT_NONE, L"No tint", RGB(255, 255, 255) },
    { IDM_TINT_FOXED_AMBER, L"Foxed Amber", RGB(215, 204, 182) },
    { IDM_TINT_BOOKCLOTH, L"Bookcloth", RGB(207, 193, 171) },
    { IDM_TINT_RECYCLED_KRAFT, L"Recycled Kraft", RGB(196, 180, 153) },
    { IDM_TINT_PLUM_KOZO, L"Plum Kozo", RGB(204, 194, 201) },
    { IDM_TINT_ROSE_QUARTZ, L"Rose Quartz", RGB(215, 199, 198) },
    { IDM_TINT_SAGE_PRESS, L"Sage Press", RGB(199, 205, 193) },
    { IDM_TINT_NORDIC_SKY, L"Nordic Sky", RGB(196, 204, 208) },

    // Paper palette. Values are approximate RGB samples from the supplied
    // reference image; the actual screen effect remains deliberately subtle.
    { IDM_TINT_SOFT_WAVE, L"Soft Wave", RGB(235, 233, 229) },
    { IDM_TINT_RICE_PAPER, L"Rice Paper", RGB(241, 239, 232) },
    { IDM_TINT_LAID_COTTON, L"Laid Cotton", RGB(232, 229, 223) },
    { IDM_TINT_NEWSPRINT, L"Newsprint", RGB(228, 226, 218) },
    { IDM_TINT_COLD_PRESS, L"Cold Press", RGB(237, 234, 227) },
    { IDM_TINT_ARTIST_CANVAS, L"Artist Canvas", RGB(234, 231, 222) },
    { IDM_TINT_FELT_SIDE, L"Felt Side", RGB(226, 223, 215) },
    { IDM_TINT_FROST_GLASSINE, L"Frost Glassine", RGB(237, 235, 230) }
};

static uint32_t Hash(uint32_t x)

```

```

{
    x ^= x >> 16;
    x *= 0x7feb352dU;
    x ^= x >> 15;
    x *= 0x846ca68bU;
    x ^= x >> 16;
    return x;
}

static void SafeDeleteLayerBitmap()
{
    if (g_memDC) {
        DeleteDC(g_memDC);
        g_memDC = nullptr;
    }

    if (g_bitmap) {
        DeleteObject(g_bitmap);
        g_bitmap = nullptr;
    }

    g_pixels = nullptr;
}

static bool CreateLayerBitmap()
{
    SafeDeleteLayerBitmap();

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    if (g_width <= 0 || g_height <= 0)
        return false;

    BITMAPINFO bi = {};
    bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
    bi.bmiHeader.biWidth = g_width;
    bi.bmiHeader.biHeight = -g_height;
    bi.bmiHeader.biPlanes = 1;
    bi.bmiHeader.biBitCount = 32;
    bi.bmiHeader.biCompression = BI_RGB;

    HDC screenDC = GetDC(nullptr);
    g_memDC = CreateCompatibleDC(screenDC);

    if (!g_memDC) {
        ReleaseDC(nullptr, screenDC);
        return false;
    }

    g_bitmap = CreateDIBSection(
        screenDC,
        &bi,
        DIB_RGB_COLORS,
        &g_pixels,
        nullptr,
        0);

    ReleaseDC(nullptr, screenDC);

    if (!g_bitmap || !g_pixels) {
        SafeDeleteLayerBitmap();
        return false;
    }

    SelectObject(g_memDC, g_bitmap);
    return true;
}

static BYTE BlendChannel(BYTE gray, BYTE tint, float amount)
{
    const float value =
        static_cast<float>(gray) * (1.0f - amount) +
        static_cast<float>(tint) * amount;

    return static_cast<BYTE>(
        std::clamp(static_cast<int>(value + 0.5f), 0, 255));
}

static void BuildLayer()
{
    if (!g_pixels || g_width <= 0 || g_height <= 0)
        return;

    auto* pixels = static_cast<Pixel*>(g_pixels);
    constexpr int TILE = 1024;

    const BYTE tintR = GetRValue(g_tint);
    const BYTE tintG = GetGValue(g_tint);
    const BYTE tintB = GetBValue(g_tint);

    // This controls how much the selected pastel affects the paper grain.
    // Kept deliberately low so the tint remains subtle.
    constexpr float TINT_MIX = 0.22f;

    for (int y = 0; y < g_height; ++y) {

```

```

    for (int x = 0; x < g_width; ++x) {
        Pixel& p = pixels[y * g_width + x];

        if (!g_enabled || g_alpha == 0) {
            p = { 0, 0, 0, 0 };
            continue;
        }

        const int tx = x & (TILE - 1);
        const int ty = y & (TILE - 1);

        const uint32_t h =
            Hash(static_cast<uint32_t>(tx) * 374761393u ^
                static_cast<uint32_t>(ty) * 668265263u);

        const float noise =
            static_cast<float>(h & 0xffffu) / 65535.0f - 0.5f;

        float value = 0.50f + noise * 0.10f;
        value = std::clamp(value, 0.25f, 0.75f);

        const BYTE gray =
            static_cast<BYTE>(value * 255.0f);

        BYTE r = gray;
        BYTE g = gray;
        BYTE b = gray;

        if (g_hasTint) {
            r = BlendChannel(gray, tintR, TINT_MIX);
            g = BlendChannel(gray, tintG, TINT_MIX);
            b = BlendChannel(gray, tintB, TINT_MIX);
        }

        // Premultiplied alpha required by UpdateLayeredWindow.
        const BYTE a = g_alpha;

        p.r = static_cast<BYTE>(
            (static_cast<unsigned int>(r) * a + 127) / 255);
        p.g = static_cast<BYTE>(
            (static_cast<unsigned int>(g) * a + 127) / 255);
        p.b = static_cast<BYTE>(
            (static_cast<unsigned int>(b) * a + 127) / 255);
        p.a = a;
    }
}

static bool PresentLayer()
{
    if (!g_memDC)
        return false;

    BuildLayer();

    HDC screenDC = GetDC(nullptr);

    POINT dst = { g_x, g_y };
    SIZE size = { g_width, g_height };
    POINT src = { 0, 0 };

    BLENDFUNCTION blend = {};
    blend.BlendOp = AC_SRC_OVER;
    blend.SourceConstantAlpha = 255;
    blend.AlphaFormat = AC_SRC_ALPHA;

    const BOOL ok = UpdateLayeredWindow(
        g_hwnd,
        screenDC,
        &dst,
        &size,
        g_memDC,
        &src,
        0,
        &blend,
        ULW_ALPHA);

    ReleaseDC(nullptr, screenDC);
    return ok != FALSE;
}

static void PositionWindow()
{
    SetWindowPos(
        g_hwnd,
        HWND_TOPMOST,
        g_x,
        g_y,
        g_width,
        g_height,
        SWP_NOACTIVATE | SWP_SHOWWINDOW);
}

static void Refresh()
{
    if (!CreateLayerBitmap())
        return;
}

```

```

    PositionWindow();
    PresentLayer();
}

static int AlphaToPercent();
static void PercentToAlpha(int percent);

static std::wstring GetSettingsPath()
{
    PWSTR appData = nullptr;
    std::wstring path;

    if (SUCCEEDED(SHGetKnownFolderPath(
        FOLDERID_RoamingAppData, 0, nullptr, &appData)) && appData) {
        path = appData;
        CoTaskMemFree(appData);

        path += L"\\GrainLayer";
        CreateDirectoryW(path.c_str(), nullptr);
        path += L"\\settings.ini";
    }

    return path;
}

static UINT GetSelectedTintId()
{
    if (!g_hasTint)
        return IDM_TINT_NONE;

    for (const auto& tint : kTints) {
        if (tint.color == g_tint)
            return tint.id;
    }

    return IDM_TINT_NONE;
}

static void SaveSettings()
{
    const std::wstring path = GetSettingsPath();
    if (path.empty())
        return;

    wchar_t buffer[32] = {};
    swprintf_s(buffer, L"%d", AlphaToPercent());
    WritePrivateProfileStringW(L"GrainLayer", L"Intensity", buffer, path.c_str());

    WritePrivateProfileStringW(
        L"GrainLayer", L"Enabled", g_enabled ? L"1" : L"0", path.c_str());

    swprintf_s(buffer, L"%u", GetSelectedTintId());
    WritePrivateProfileStringW(L"GrainLayer", L"Tint", buffer, path.c_str());
}

static void LoadSettings()
{
    const std::wstring path = GetSettingsPath();
    if (path.empty())
        return;

    wchar_t buffer[32] = {};

    GetPrivateProfileStringW(
        L"GrainLayer", L"Intensity", L"25", buffer, _countof(buffer), path.c_str());
    PercentToAlpha(_wtoi(buffer));

    GetPrivateProfileStringW(
        L"GrainLayer", L"Enabled", L"1", buffer, _countof(buffer), path.c_str());
    g_enabled = (_wtoi(buffer) != 0);

    GetPrivateProfileStringW(
        L"GrainLayer", L"Tint", L"1100", buffer, _countof(buffer), path.c_str());
    const UINT tintId = static_cast<UINT>(_wtoi(buffer));

    if (tintId == IDM_TINT_NONE) {
        g_tint = RGB(255, 255, 255);
        g_hasTint = false;
    } else {
        const TintMenuItem* tint = nullptr;
        for (const auto& item : kTints) {
            if (item.id == tintId) {
                tint = &item;
                break;
            }
        }

        if (tint) {
            g_tint = tint->color;
            g_hasTint = true;
        }
    }
}

static bool IsStartupEnabled()
{
    HKEY key = nullptr;
    if (RegOpenKeyExW(

```

```

        HKEY_CURRENT_USER,
        L"Software\\Microsoft\\Windows\\CurrentVersion\\Run",
        0,
        KEY_QUERY_VALUE,
        &key) != ERROR_SUCCESS)
    return false;

wchar_t value[1024] = {};
DWORD type = 0;
DWORD size = sizeof(value);
const LONG result = RegQueryValueExW(
    key, L"GrainLayer", nullptr, &type,
    reinterpret_cast<LPBYTE>(value), &size);
RegCloseKey(key);

return result == ERROR_SUCCESS &&
    (type == REG_SZ || type == REG_EXPAND_SZ) &&
    value[0] != L'\0';
}

static bool SetStartupEnabled(bool enabled)
{
    HKEY key = nullptr;
    const LONG openResult = RegOpenKeyExW(
        HKEY_CURRENT_USER,
        L"Software\\Microsoft\\Windows\\CurrentVersion\\Run",
        0,
        KEY_SET_VALUE,
        &key);

    if (openResult != ERROR_SUCCESS)
        return false;

    LONG result = ERROR_SUCCESS;

    if (enabled) {
        wchar_t exePath[MAX_PATH] = {};
        if (GetModuleFileNameW(nullptr, exePath, _countof(exePath)) == 0) {
            result = GetLastError();
        } else {
            std::wstring command = L"";
            command += exePath;
            command += L"\\";

            result = RegSetValueExW(
                key,
                L"GrainLayer",
                0,
                REG_SZ,
                reinterpret_cast<const BYTE*>(command.c_str()),
                static_cast<DWORD>((command.size() + 1) * sizeof(wchar_t)));
        }
    } else {
        result = RegDeleteValueW(key, L"GrainLayer");
        if (result == ERROR_FILE_NOT_FOUND)
            result = ERROR_SUCCESS;
    }

    RegCloseKey(key);
    return result == ERROR_SUCCESS;
}

static void ToggleOverlay()
{
    g_enabled = !g_enabled;
    PresentLayer();
    SaveSettings();
}

static void ChangeOpacity(int delta)
{
    g_alpha = static_cast<BYTE>(
        std::clamp(static_cast<int>(g_alpha) + delta, 0, 100));
    PresentLayer();

    if (g_menuTracking)
        RefreshPopupMenu();

    SaveSettings();
}

static void SetTint(COLORREF color, bool enabled)
{
    g_tint = color;
    g_hasTint = enabled;
    PresentLayer();
    SaveSettings();
}

static void AddTrayIcon()
{
    ZeroMemory(&g_nid, sizeof(g_nid));

    g_nid.cbSize = sizeof(g_nid);
    g_nid.hWnd = g_hwnd;
    g_nid.uID = 1;
    g_nid.uFlags = NIF_MESSAGE | NIF_ICON | NIF_TIP;
    g_nid.uCallbackMessage = WMAPP_TRAY;
}

```

```

    g_nid.hIcon = LoadIconW(GetModuleHandleW(nullptr), MAKEINTRESOURCEW(IDI_APP_ICON));

    wcscpy_s(g_nid.szTip, L"GrainLayer");

    Shell_NotifyIconW(NIM_ADD, &g_nid);
}

static void RemoveTrayIcon()
{
    Shell_NotifyIconW(NIM_DELETE, &g_nid);
}

static const TintMenuItem* FindTint(UINT id)
{
    for (const auto& tint : kTints) {
        if (tint.id == id)
            return &tint;
    }
    return nullptr;
}

static void AppendTintItem(HMENU menu, const TintMenuItem& tint)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = tint.id;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData =
        reinterpret_cast<ULONG_PTR>(&tint);

    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

static int AlphaToPercent()
{
    return (static_cast<int>(g_alpha) * 100 + 127) / 255;
}

static void PercentToAlpha(int percent)
{
    percent = std::clamp(percent, 0, 100);
    g_alpha = static_cast<BYTE>((percent * 255 + 50) / 100);
}

static void SetIntensityFromCursor(HMENU menu)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetMenuItemRect(g_hwnd, menu, IDM_INTENSITY_SLIDER, &itemRect))
        return;

    POINT pt = {};
    GetCursorPos(&pt);

    const int left = itemRect.left + 28;
    const int right = itemRect.right - 18;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);
    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha)
        PresentLayer();
}

static void AppendIntensitySlider(HMENU menu)
{
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID | MIIM_FTYPE | MIIM_DATA;
    mii.wID = IDM_INTENSITY_SLIDER;
    mii.fType = MFT_OWNERDRAW;
    mii.dwItemData = 0;
    InsertMenuItemW(
        menu,
        GetMenuItemCount(menu),
        TRUE,
        &mii);
}

```

```

}

static void DrawIntensitySlider(const DRAWITEMSTRUCT* dis)
{
    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const COLORREF bg = GetSysColor(COLOR_MENU);
    const COLORREF text = GetSysColor(COLOR_MENUTEXT);
    const COLORREF face = GetSysColor(COLOR_3DFACE);
    const COLORREF shadow = GetSysColor(COLOR_3DSHADOW);

    HBRUSH bgBrush = CreateSolidBrush(bg);
    FillRect(dc, &rc, bgBrush);
    DeleteObject(bgBrush);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT label = rc;
    label.left += 8;
    label.top += 3;
    label.right -= 8;
    label.bottom = label.top + 18;

    wchar_t textBuffer[64] = {};
    swprintf_s(
        textBuffer,
        L"Intensity",
        AlphaToPercent(),
        %d%%",

    DrawTextW(
        dc,
        textBuffer,
        -1,
        &label,
        DT_SINGLELINE | DT_LEFT | DT_VCENTER | DT_NOPREFIX);

    const int trackLeft = rc.left + 12;
    const int trackRight = rc.right - 12;
    const int trackY = rc.bottom - 14;
    const int trackHeight = 4;

    RECT track = {
        trackLeft,
        trackY - trackHeight / 2,
        trackRight,
        trackY + (trackHeight + 1) / 2
    };

    HBRUSH trackBrush = CreateSolidBrush(shadow);
    FillRect(dc, &track, trackBrush);
    DeleteObject(trackBrush);

    const int percent = AlphaToPercent();
    const int knobX =
        trackLeft +
        ((trackRight - trackLeft) * percent) / 100;

    RECT filled = track;
    filled.right = knobX;

    HBRUSH fillBrush = CreateSolidBrush(
        GetSysColor(COLOR_HIGHLIGHT));
    FillRect(dc, &filled, fillBrush);
    DeleteObject(fillBrush);

    const int radius = 7;
    RECT knob = {
        knobX - radius,
        trackY - radius,
        knobX + radius + 1,
        trackY + radius + 1
    };

    HBRUSH knobBrush = CreateSolidBrush(face);
    FillRect(dc, &knob, knobBrush);
    DeleteObject(knobBrush);

    FrameRect(
        dc,
        &knob,
        static_cast<HBRUSH>(GetStockObject(GRAY_BRUSH)));
}

static void MeasureIntensitySlider(MEASUREITEMSTRUCT* mis)
{
    mis->itemHeight = 48;
    mis->itemWidth = 250;
}

static HMENU g_activeMenu = nullptr;
static HHOOK g_menuHook = nullptr;
static bool g_sliderDragging = false;

static bool GetIntensityItemRect(HMENU menu, RECT* itemRect)
{
    if (!menu || !itemRect)

```

```

        return false;

const int count = GetMenuItemCount(menu);

for (int position = 0; position < count; ++position) {
    MENUITEMINFOW mii = {};
    mii.cbSize = sizeof(mii);
    mii.fMask = MIIM_ID;

    if (GetMenuItemInfo(
        menu,
        static_cast<UINT>(position),
        TRUE,
        &mii) &&
        mii.wID == IDM_INTENSITY_SLIDER) {

        return GetMenuItemRect(
            g_hwnd,
            menu,
            static_cast<UINT>(position),
            itemRect) != FALSE;
    }
}

return false;
}

static HWND FindPopupMenuWindow()
{
    // Standard Win32 popup menus are implemented by the internal #32768
    // window class. This is used only to force an immediate repaint of the
    // owner-drawn slider while TrackPopupMenu is running.
    return FindWindowExW(
        nullptr,
        nullptr,
        L"#32768",
        nullptr);
}

static void RefreshPopupMenu()
{
    HWND menuWindow = FindPopupMenuWindow();
    if (menuWindow) {
        InvalidateRect(menuWindow, nullptr, FALSE);
        UpdateWindow(menuWindow);
    }
}

static void SetIntensityFromPoint(HMENU menu, POINT pt)
{
    if (!menu)
        return;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return;

    const int left = itemRect.left + 12;
    const int right = itemRect.right - 12;

    if (right <= left)
        return;

    const int mouseX = static_cast<int>(pt.x);

    const int clampedX =
        mouseX < left ? left :
        (mouseX > right ? right : mouseX);

    const int percent =
        ((clampedX - left) * 100 + (right - left) / 2) /
        (right - left);

    const BYTE oldAlpha = g_alpha;
    PercentToAlpha(percent);

    if (oldAlpha != g_alpha) {
        PresentLayer();
        RefreshPopupMenu();
        SaveSettings();
    }
}

static bool IsPointOverSlider(HMENU menu, POINT pt)
{
    if (!menu)
        return false;

    RECT itemRect = {};
    if (!GetIntensityItemRect(menu, &itemRect))
        return false;

    return PtInRect(&itemRect, pt) != FALSE;
}

static LRESULT CALLBACK MenuMessageFilter(
    int code,
    WPARAM wParam,

```

```

LPARAM lParam)
{
    if (code == MSGF_MENU && g_activeMenu) {
        const MSG* msg =
            reinterpret_cast<const MSG*>(lParam);

        if (msg) {
            switch (msg->message) {
                case WM_LBUTTONDOWN:
                    if (IsPointOverSlider(g_activeMenu, msg->pt)) {
                        g_sliderDragging = true;

                        // Keep receiving mouse movement even when the pointer
                        // leaves the slider while the button is held.
                        HWND menuWindow = FindPopupMenuWindow();
                        if (menuWindow)
                            SetCapture(menuWindow);

                        SetIntensityFromPoint(g_activeMenu, msg->pt);

                        // Consume the click so the menu stays open.
                        return 1;
                    }
                    break;

                case WM_MOUSEMOVE:
                    // A plain mouseover must not change the intensity.
                    // Only a left-button drag updates the slider.
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        return 1;
                    }
                    break;

                case WM_LBUTTONUP:
                    if (g_sliderDragging) {
                        SetIntensityFromPoint(g_activeMenu, msg->pt);
                        g_sliderDragging = false;

                        if (GetCapture() == FindPopupMenuWindow())
                            ReleaseCapture();

                        // Consume the release so the menu stays open.
                        return 1;
                    }
                    break;
            }
        }
    }

    return CallNextHookEx(g_menuHook, code, wParam, lParam);
}

static void ShowTrayMenu()
{
    HMENU menu = CreatePopupMenu();
    if (!menu)
        return;

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_TOGGLE,
        g_enabled ? L"Toggle overlay (F6)" : L"Toggle overlay (F6)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_DOWN,
        L"Decrease intensity (F7)");

    AppendMenuW(
        menu,
        MF_STRING,
        IDM_OPACITY_UP,
        L"Increase intensity (F8)");

    AppendIntensitySlider(menu);

    AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

    AppendMenuW(menu, MF_STRING, IDM_TINT_NONE, L"No tint");

    HMENU warmMenu = CreatePopupMenu();
    if (warmMenu) {
        AppendTintItem(warmMenu, kTints[1]);
        AppendTintItem(warmMenu, kTints[2]);
        AppendTintItem(warmMenu, kTints[3]);
        AppendTintItem(warmMenu, kTints[4]);
        AppendTintItem(warmMenu, kTints[5]);
        AppendTintItem(warmMenu, kTints[6]);
        AppendTintItem(warmMenu, kTints[7]);

        AppendMenuW(
            menu,
            MF_POPUP,
            reinterpret_cast<UINT_PTR>(warmMenu),
            L"Warm & tinted");
    }
}

```

```

}

HMENU paperMenu = CreatePopupMenu();
if (paperMenu) {
    AppendTintItem(paperMenu, kTints[8]);
    AppendTintItem(paperMenu, kTints[9]);
    AppendTintItem(paperMenu, kTints[10]);
    AppendTintItem(paperMenu, kTints[11]);
    AppendTintItem(paperMenu, kTints[12]);
    AppendTintItem(paperMenu, kTints[13]);
    AppendTintItem(paperMenu, kTints[14]);
    AppendTintItem(paperMenu, kTints[15]);

    AppendMenuW(
        menu,
        MF_POPUP,
        reinterpret_cast<UINT_PTR>(paperMenu),
        L"Paper");
}

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(
    menu,
    MF_STRING | (IsStartupEnabled() ? MF_CHECKED : 0),
    IDM_STARTUP,
    L"Open at startup");

AppendMenuW(menu, MF_SEPARATOR, 0, nullptr);

AppendMenuW(
    menu,
    MF_STRING,
    IDM_EXIT,
    L"Exit (Ctrl+Alt+Shift+Q)");

POINT pt = {};
GetCursorPos(&pt);

SetForegroundWindow(g_hwnd);

g_activeMenu = menu;
g_menuTracking = true;
g_sliderDragging = false;

g_menuHook = SetWindowsHookExW(
    WH_MSGFILTER,
    MenuMessageFilter,
    nullptr,
    GetCurrentThreadId());

TrackPopupMenu(
    menu,
    TPM_RIGHTBUTTON |
    TPM_BOTTOMALIGN |
    TPM_LEFTALIGN,
    pt.x,
    pt.y,
    0,
    g_hwnd,
    nullptr);

if (g_menuHook) {
    UnhookWindowsHookEx(g_menuHook);
    g_menuHook = nullptr;
}

g_sliderDragging = false;
g_menuTracking = false;
g_activeMenu = nullptr;

DestroyMenu(menu);
}

static void DrawTintSwatch(
    HDC dc,
    const RECT& rc,
    COLORREF color)
{
    RECT swatch = rc;

    swatch.left += 8;
    swatch.top += 4;
    swatch.right = swatch.left + 16;
    swatch.bottom = swatch.top + 16;

    HBRUSH fill = CreateSolidBrush(color);
    FillRect(dc, &swatch, fill);
    DeleteObject(fill);

    FrameRect(
        dc,
        &swatch,
        static_cast<HBRUSH>(
            GetStockObject(GRAY_BRUSH)));
}

static void DrawTintMenuItem(const DRAWITEMSTRUCT* dis)

```

```

{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            dis->itemData);

    if (!tint)
        return;

    HDC dc = dis->hDC;
    RECT rc = dis->rcItem;

    const bool selected =
        (dis->itemState & ODS_SELECTED) != 0;

    const COLORREF background =
        selected
        ? GetSysColor(COLOR_HIGHLIGHT)
        : GetSysColor(COLOR_MENU);

    const COLORREF text =
        selected
        ? GetSysColor(COLOR_HIGHLIGHTTEXT)
        : GetSysColor(COLOR_MENUTEXT);

    HBRUSH bg = CreateSolidBrush(background);
    FillRect(dc, &rc, bg);
    DeleteObject(bg);

    DrawTintSwatch(dc, rc, tint->color);

    SetBkMode(dc, TRANSPARENT);
    SetTextColor(dc, text);

    RECT textRc = rc;
    textRc.left += 34;
    textRc.right -= 8;

    DrawTextW(
        dc,
        tint->name,
        -1,
        &textRc,
        DT_SINGLELINE |
        DT_VCENTER |
        DT_LEFT |
        DT_NOPREFIX);
}

static void MeasureTintMenuItem(MEASUREITEMSTRUCT* mis)
{
    const auto* tint =
        reinterpret_cast<const TintMenuItem*>(
            mis->itemData);

    if (!tint)
        return;

    mis->itemHeight = 24;

    HDC dc = GetDC(g_hwnd);
    HFONT font = static_cast<HFONT>(
        GetStockObject(DEFAULT_GUI_FONT));

    HFONT old = static_cast<HFONT>(
        SelectObject(dc, font));

    SIZE size = {};
    GetTextExtentPoint32W(
        dc,
        tint->name,
        lstrlenW(tint->name),
        &size);

    SelectObject(dc, old);
    ReleaseDC(g_hwnd, dc);

    mis->itemWidth =
        static_cast<UINT>(size.cx + 50);
}

static LRESULT CALLBACK WndProc(
    HWND h,
    UINT msg,
    WPARAM wParam,
    LPARAM lParam)
{
    switch (msg) {
    case WM_APP_TRAY:
        if (lParam == WM_RBUTTONDOWN)
            ShowTrayMenu();
        else if (lParam == WM_LBUTTONDOWNCLK)
            ToggleOverlay();
        return 0;

    case WM_MEASUREITEM:
        if (wParam == 0) {
            const auto* mis =
                reinterpret_cast<const MEASUREITEMSTRUCT*>(lParam);

```

```

        if (mis && mis->CtlType == ODT_MENU) {
            auto* mutableMis =
                const_cast<MEASUREITEMSTRUCT*>(mis);

            if (mis->itemID == IDM_INTENSITY_SLIDER) {
                MeasureIntensitySlider(mutableMis);
                return TRUE;
            }

            if (FindTint(mis->itemID)) {
                MeasureTintMenuItem(mutableMis);
                return TRUE;
            }
        }
    }
    break;

case WM_DRAWITEM:
    if (wParam == 0) {
        const auto* dis =
            reinterpret_cast<const DRAWITEMSTRUCT*>(lParam);

        if (dis && dis->CtlType == ODT_MENU) {
            if (dis->itemID == IDM_INTENSITY_SLIDER) {
                DrawIntensitySlider(dis);
                return TRUE;
            }

            if (FindTint(dis->itemID)) {
                DrawTintMenuItem(dis);
                return TRUE;
            }
        }
    }
    break;

case WM_COMMAND:
    switch (LOWORD(wParam)) {
        case IDM_TOGGLE:
            ToggleOverlay();
            return 0;

        case IDM_OPACITY_DOWN:
            ChangeOpacity(-5);
            return 0;

        case IDM_OPACITY_UP:
            ChangeOpacity(+5);
            return 0;

        case IDM_STARTUP:
            SetStartupEnabled(!IsStartupEnabled());
            return 0;

        case IDM_EXIT:
            DestroyWindow(h);
            return 0;

        case IDM_TINT_NONE:
            SetTint(RGB(255, 255, 255), false);
            return 0;

        default:
        {
            const TintMenuItem* tint =
                FindTint(LOWORD(wParam));

            if (tint) {
                SetTint(tint->color, true);
                return 0;
            }
        }
    }
    break;
}
break;

case WM_NCHITTEST:
    return HTTRANSPARENT;

case WM_MOUSEACTIVATE:
    return MA_NOACTIVATE;

case WM_DISPLAYCHANGE:
    Refresh();
    return 0;

case WM_HOTKEY:
    switch (wParam) {
        case 1:
            ToggleOverlay();
            break;

        case 2:
            ChangeOpacity(-5);
            break;

        case 3:
            ChangeOpacity(+5);
    }
}

```

```

        break;

    case 4:
        DestroyWindow(h);
        break;
    }
    return 0;

case WM_DESTROY:
    SaveSettings();
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(h, msg, wParam, lParam);
}

int WINAPI wWinMain(
    HINSTANCE instance,
    HINSTANCE,
    PWSTR,
    int)
{
    INITCOMMONCONTROLSEX icc = {};
    icc.dwSize = sizeof(icc);
    icc.dwICC = ICC_BAR_CLASSES;
    InitCommonControlsEx(&icc);

    WNDCLASSEXW wc = {};
    wc.cbSize = sizeof(wc);
    wc.hInstance = instance;
    wc.lpfnWndProc = WndProc;
    wc.lpszClassName = L"GrainLayerOverlay";
    wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wc.hIcon = LoadIconW(instance, MAKEINTRESOURCEW(IDI_APP_ICON));
    wc.hIconSm = LoadIconW(instance, MAKEINTRESOURCEW(IDI_APP_ICON));

    if (!RegisterClassExW(&wc))
        return 1;

    g_x = GetSystemMetrics(SM_XVIRTUALSCREEN);
    g_y = GetSystemMetrics(SM_YVIRTUALSCREEN);
    g_width = GetSystemMetrics(SM_CXVIRTUALSCREEN);
    g_height = GetSystemMetrics(SM_CYVIRTUALSCREEN);

    const DWORD exStyle =
        WS_EX_LAYERED |
        WS_EX_TRANSPARENT |
        WS_EX_NOACTIVATE |
        WS_EX_TOOLWINDOW |
        WS_EX_TOPMOST;

    g_hwnd = CreateWindowExW(
        exStyle,
        wc.lpszClassName,
        L"GrainLayer",
        WS_POPUP,
        g_x,
        g_y,
        g_width,
        g_height,
        nullptr,
        nullptr,
        instance,
        nullptr);

    if (!g_hwnd)
        return 2;

    LoadSettings();

    if (!CreateLayerBitmap())
        return 3;

    AddTrayIcon();

    RegisterHotKey(
        g_hwnd, 1, MOD_NOREPEAT, VK_F6);

    RegisterHotKey(
        g_hwnd, 2, MOD_NOREPEAT, VK_F7);

    RegisterHotKey(
        g_hwnd, 3, MOD_NOREPEAT, VK_F8);

    RegisterHotKey(
        g_hwnd,
        4,
        MOD_CONTROL |
        MOD_ALT |
        MOD_SHIFT |
        MOD_NOREPEAT,
        'Q');

    ShowWindow(g_hwnd, SW_SHOWNOACTIVATE);
    PositionWindow();
    PresentLayer();
}

```

```

MSG msg = {};
while (GetMessageW(&msg, nullptr, 0, 0) > 0) {
    TranslateMessage(&msg);
    DispatchMessageW(&msg);
}

UnregisterHotKey(g_hwnd, 1);
UnregisterHotKey(g_hwnd, 2);
UnregisterHotKey(g_hwnd, 3);
UnregisterHotKey(g_hwnd, 4);

RemoveTrayIcon();
SafeDeleteLayerBitmap();

return 0;
}

```

11. Appendix B --- Historical Version Map

Version Name Main milestone ----- 0.1 Deckle Windows First Windows overlay experiment

0.2 Deckle Windows Overlay refinement

0.3 Deckle Windows Per-pixel alpha and UpdateLayeredWindow

0.4 Deckle Windows Additional hotkey/control refinement

0.5 Deckle Windows System tray and unified opacity controls

0.6 Deckle Windows Stabilization

0.7 --- No preserved source snapshot in the archive

0.8 GrainLayer Feature-oriented GrainLayer line

0.9 GrainLayer Stabilization

1.0 GrainLayer Stable Visual Studio project structure

1.0.1 GrainLayer Bridge to graphical intensity control

1.1 GrainLayer Graphical intensity control

1.2 GrainLayer Owner-drawn tray-menu slider

1.2.1 GrainLayer Slider refinement

1.2.2 GrainLayer Message-filtered live dragging

1.2.3 GrainLayer Syntax/source repair

1.2.4 GrainLayer Correct menu-position hit-testing

1.2.5 GrainLayer Live click-and-drag

1.2.6 GrainLayer Natural hover/click/drag semantics

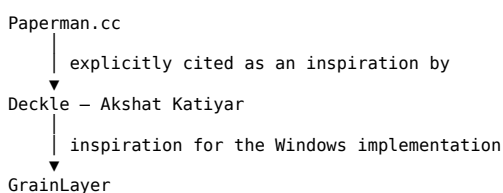
1.3 GrainLayer Embedded custom icon

1.4 GrainLayer Persistent settings and Windows startup

1.4.1 GrainLayer Correct F7/F8 direction and slider synchronization

12. Appendix C --- Credits and Inspiration

GrainLayer belongs to a small lineage of desktop paper-surface projects. The relationship is best represented as:



12.1. Paperman.cc

[Paperman.cc](https://paperman.cc/) is a desktop paper-surface utility for Mac and Windows. It is the earlier project in this particular chain of inspiration.

12.2. Deckle

[Deckle](https://projects.akshatkatiyar.com/projects/deckle/) is a macOS application created by **Akshat Katiyar**. Deckle explicitly cites Paperman as an inspiration. GrainLayer takes Deckle as its immediate inspiration for an independent Windows implementation.

The chain is therefore:

Paperman.cc → Deckle → GrainLayer

GrainLayer is an independent Windows project. It is not affiliated with, endorsed by, or an official Windows port of Deckle or Paperman.

Special thanks to Akshat Katiyar for Deckle and its inspiration, and to the creators of Paperman.cc for the earlier project in this lineage.

The project was developed as an experiment in AI-assisted programming / vibe coding using ChatGPT. The code was generated and revised through an iterative process of requirements, source review, compilation, testing, bug reports and corrections.

That development process is itself part of what this manual documents.

13. Appendix D --- Final Notes for the Curious Programmer

GrainLayer is intentionally small enough to remain readable.

It does not use a GUI framework. It does not use a rendering engine. It does not need a background service. It does not need administrator privileges for normal operation.

Instead, it relies on a handful of old and extremely durable Windows mechanisms:

```
WinMain / wWinMain
+
Win32 message loop
+
layered window
+
32-bit DIB
+
GDI
+
Shell_NotifyIcon
+
owner-drawn menu
+
registry / INI persistence
```

That combination is enough to make a modern-feeling desktop utility.

The most useful lesson of the entire project is therefore not any particular function call. It is the process:

make one thing work → observe the failure → understand the Windows mechanism → change one layer → test again → keep the working parts stable.

That is how a 248-line experiment called Deckle Windows eventually became GrainLayer.

NOTE: The screenshots are part of the documentation

Figures 1–3 show the finished v1.4.1 application. The first two are the same desktop with the overlay disabled and enabled; the third shows the tray menu that acts as the application's main control surface.